

电子设计基础训练

——Arduino串口通信实验

北京航空航天大学
电子信息工程学院



实验目的

■ 认识基本通信系统

- 发送与接收
- 搭建简单有线通信系统

■ 熟悉Arduino的串口通信

- 开发板与计算机通信
- 开发板与开发板通信

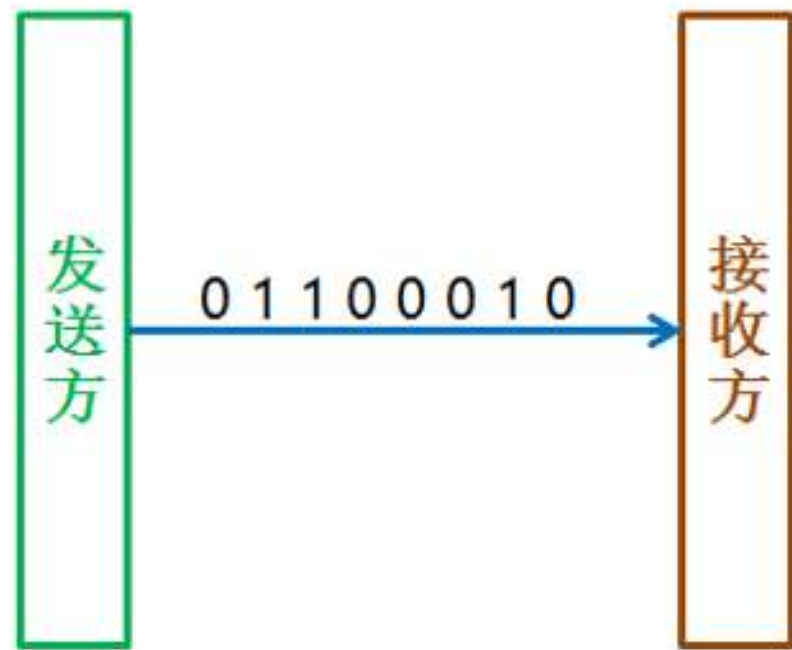
■ 掌握如何通过串口监视器辅助Arduino系统调试



串行通信

■ 串行通信概念介绍

- 串行通信是一种逐位传输数据的通信方式，数据通过单一信道（一根导线或者无线信道）按顺序发送。
- 优点：传输线少，抗干扰能力强适合长距离传输。
- 缺点：传输协议较为复杂。

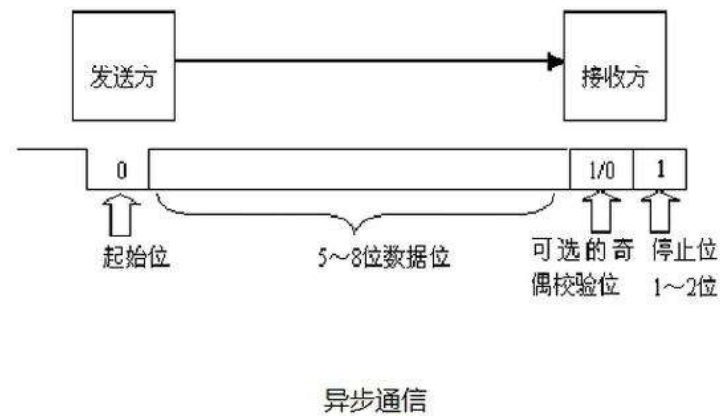
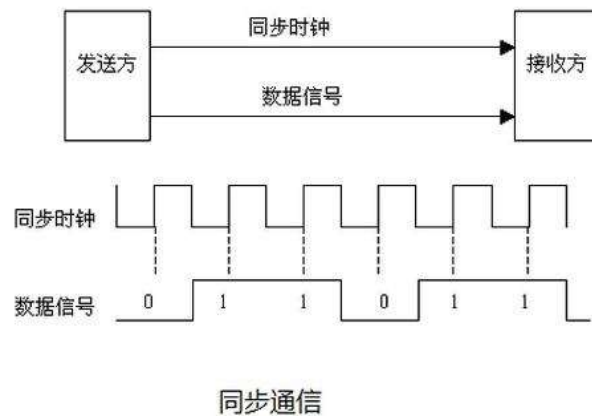




串行通信

■ 串行通信分类：

- **同步通信：**发送方和接收方使用同一个时钟信号。
- **异步通信：**发送方和接收方使用各自的时钟信号，按照约定的波特率发送数据。

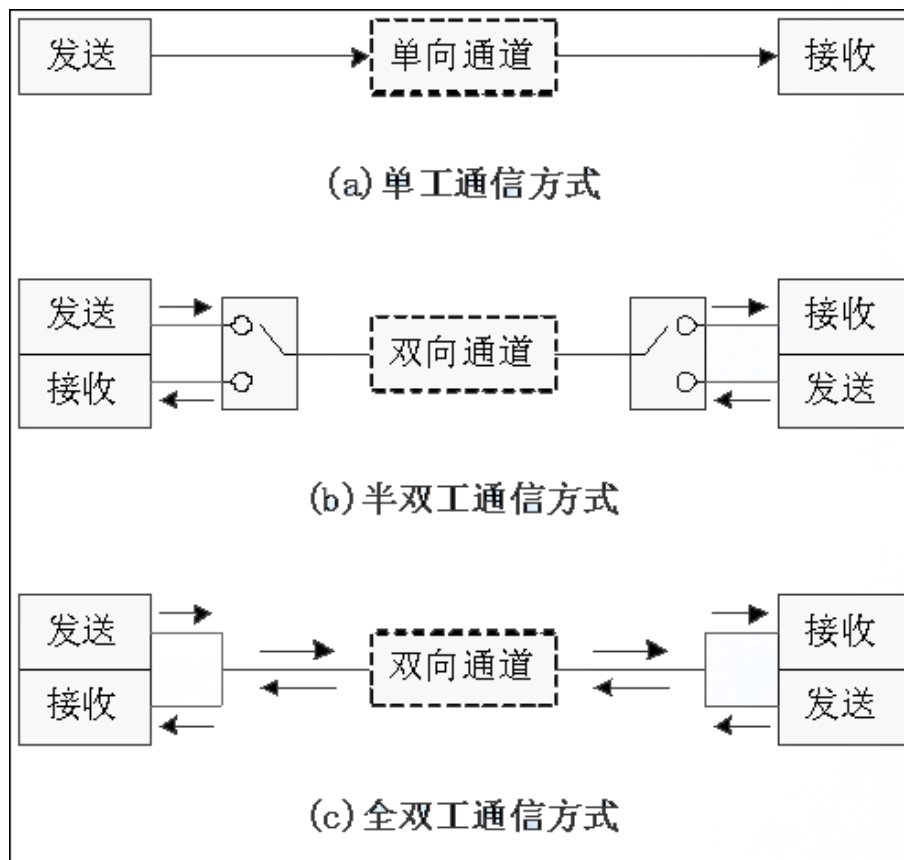




串行通信

■ 串行通信按传输方向分类：

- **单工**：是指数据传输仅能沿一个方向，不能实现反向传输。
- **半双工**：是指数据传输可以沿两个方向，但同一时间只能单向传输。
- **全双工**：是指数据可以同时进行双向传输。
- **Arduino**采用通用异步接收器/发送器(UART)通信设计，是一种**全双工、异步**的通信方式。





串行通信

■ 串行通信

妈，我**不**回家
吃饭了



'0'





串行通信

妈，我回家吃饭了



'1'





串行通信

不 = '0'
外面 = '010'
烩面 = '0110'

妈，我**不**回家
我在**外面**吃**烩面**



“?”





串行通信



起始位

停止位

何时开始?

何时结束?





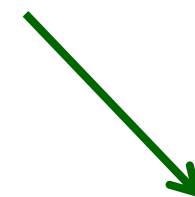
串行通信

■ 奇偶校验

=> **奇校验**—— '1' 的总个数是奇数

0 0 1 0 0 1 1 0

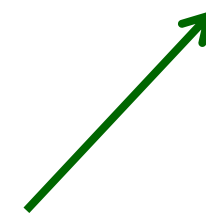
0



=> **偶校验**—— '1' 的总个数是偶数

0 0 1 0 0 1 1 0

1



校验位



串行通信

■ 奇偶校验

=> 有限错误检测码

奇校验

0 0 1 0 0 1 1 0 [0]

0 0 1 0 [1] 1 1 0 [0]

数据无效!

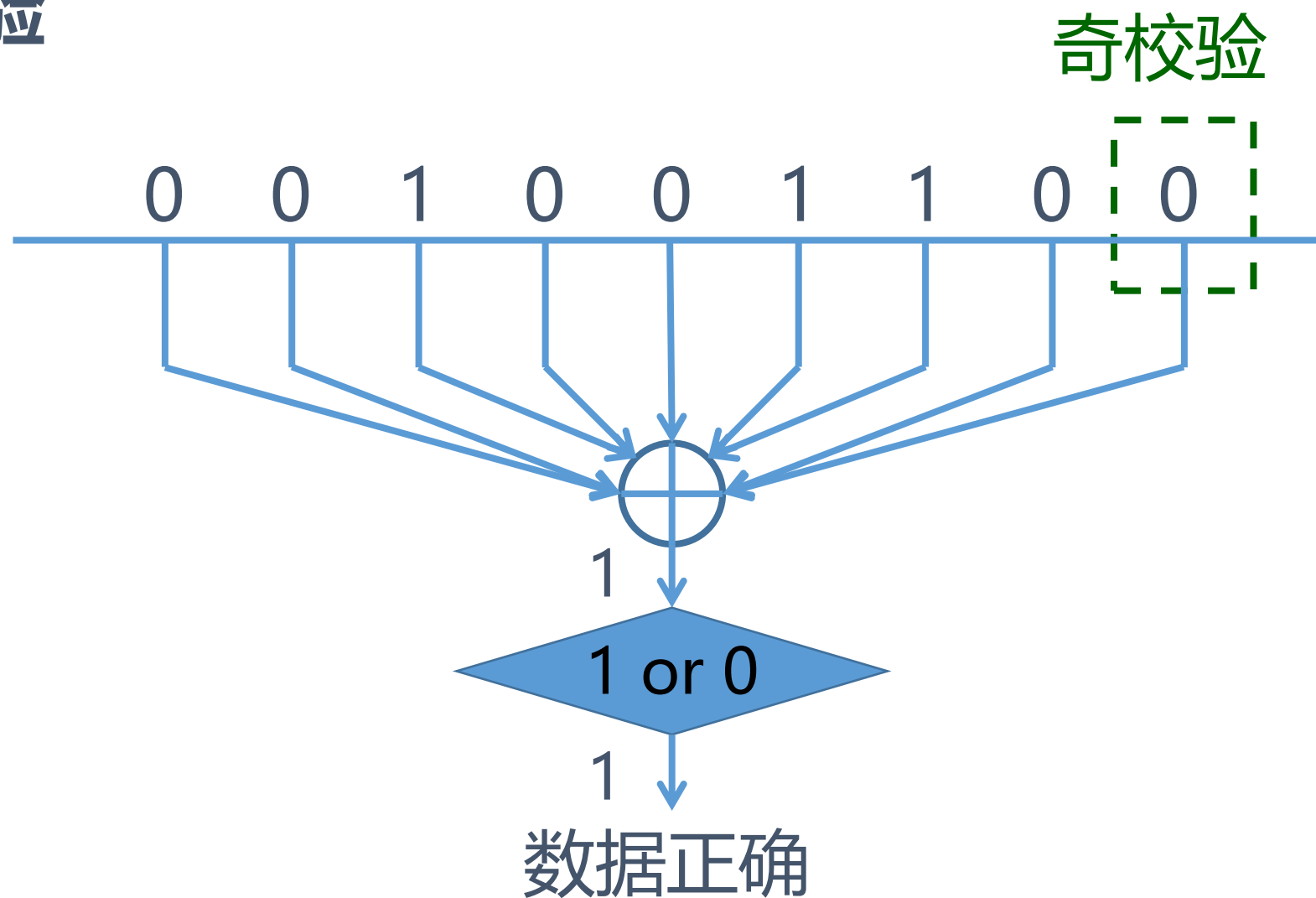
0 0 1 0 [1] 1 [0] 0 [0]

无法检测!



串行通信

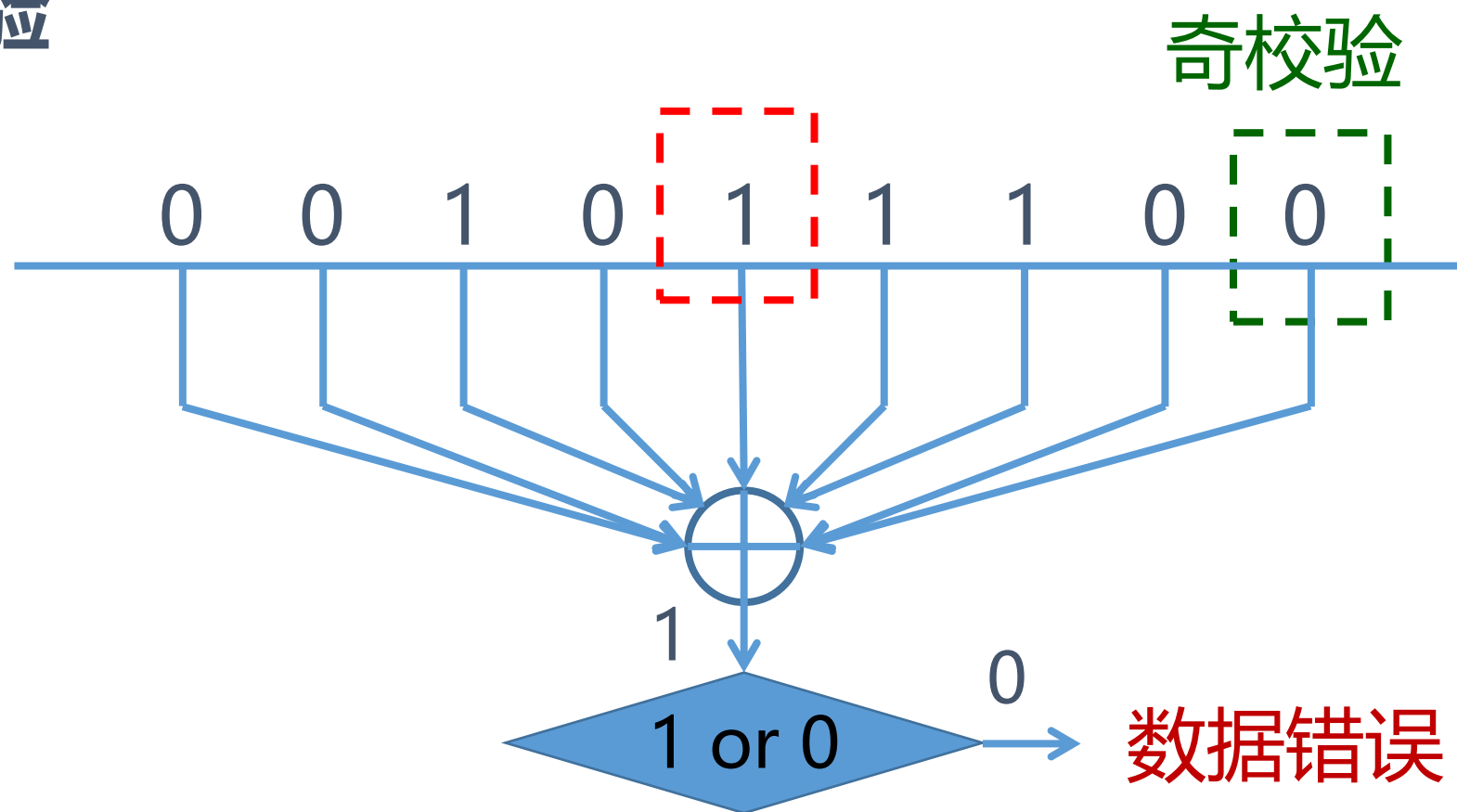
■ 奇偶校验





串行通信

■ 奇偶校验





串行通信

■ 串行通信



第一帧: 0 0 1 0 0 1 1 0



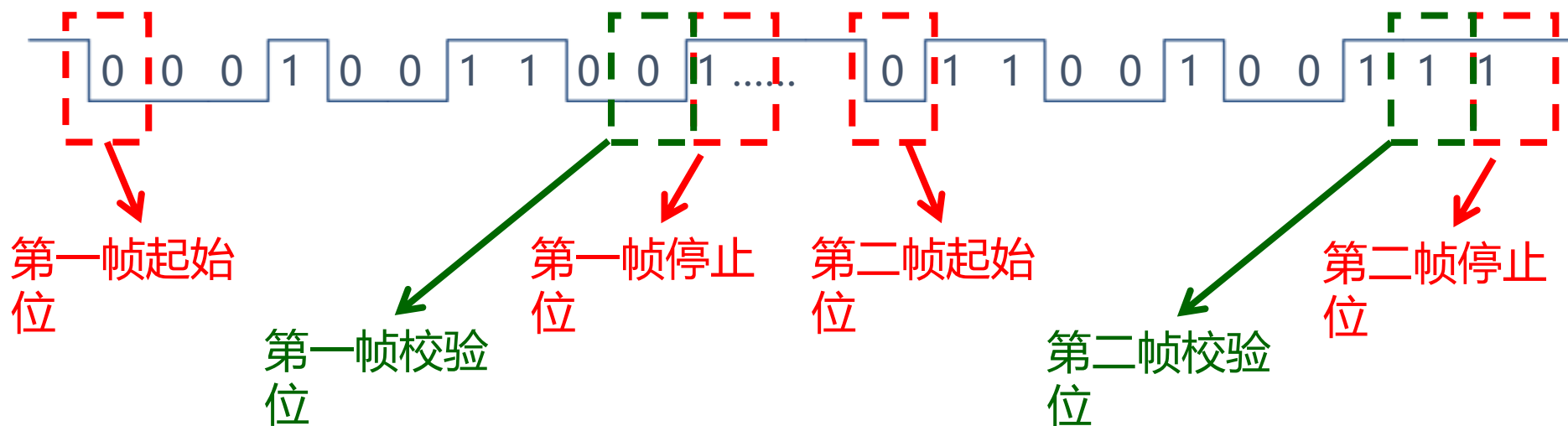


串行通信

■ 串行通信



第一帧: 0 0 1 0 0 1 1 0





串行通信

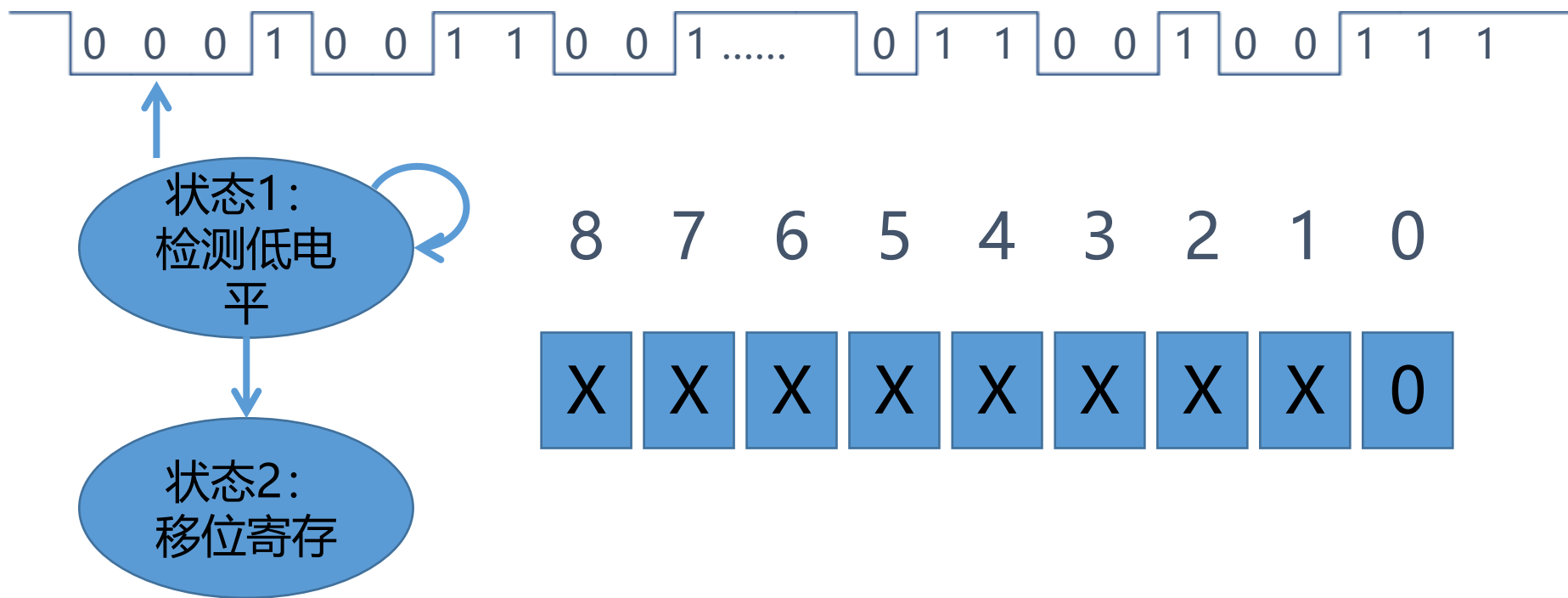
■ 接收端如何工作





串行通信

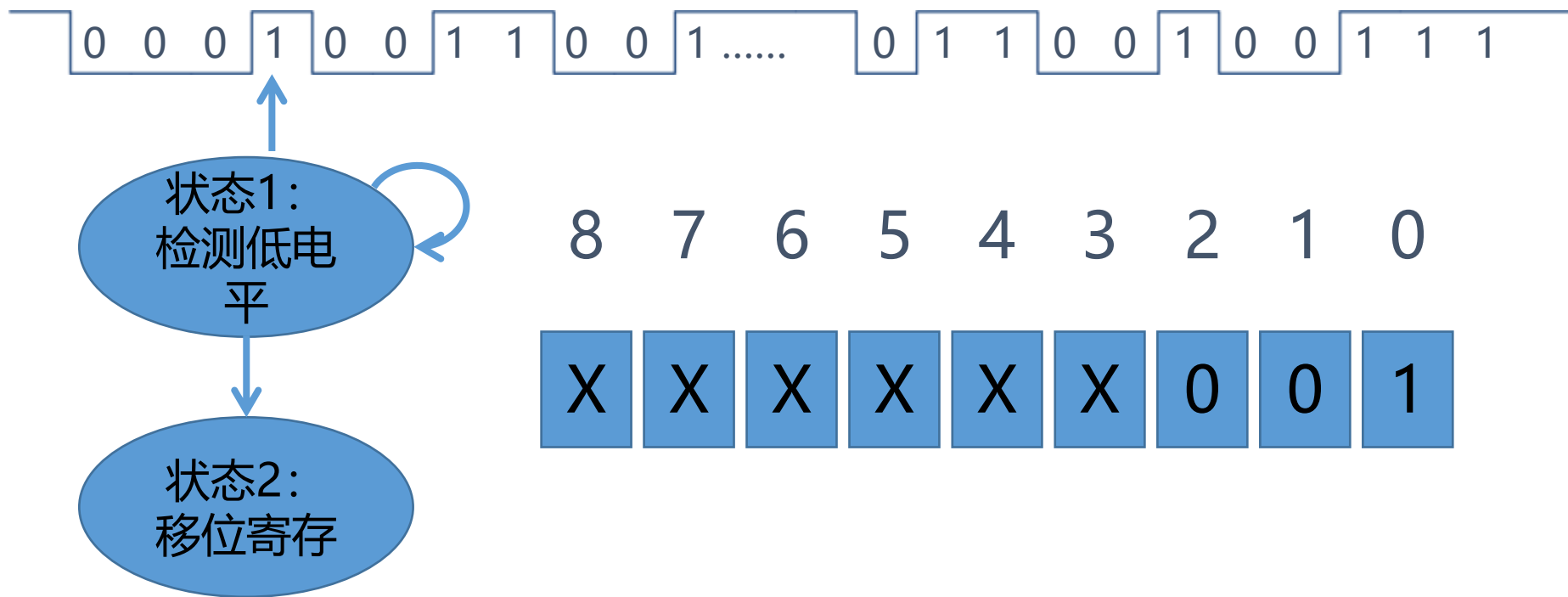
■ 接收端如何工作





串行通信

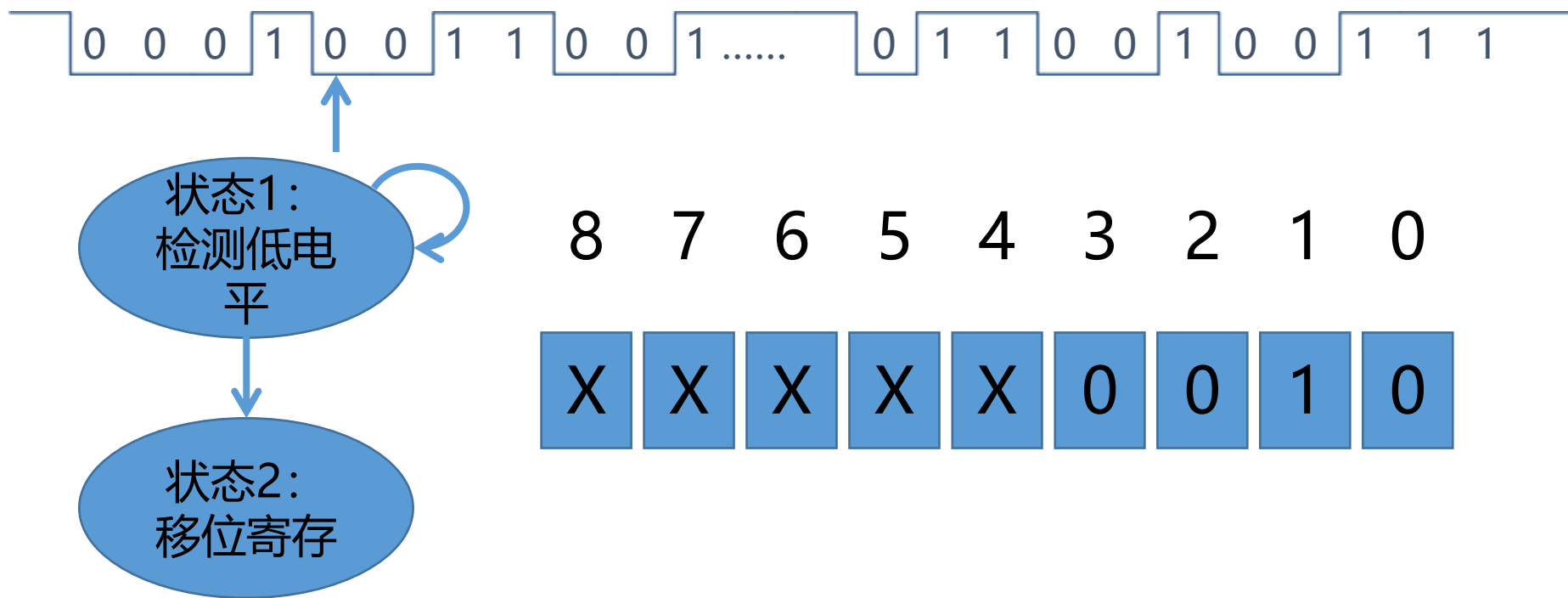
■ 接收端如何工作





串行通信

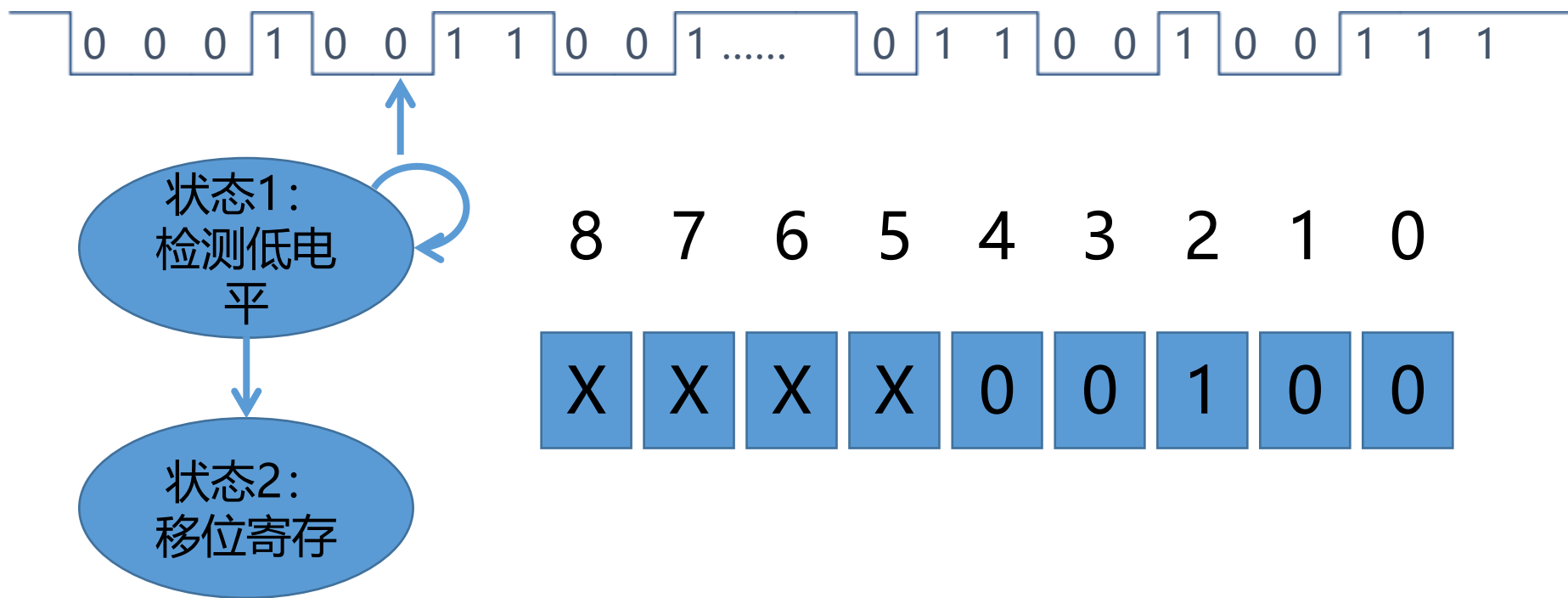
■ 接收端如何工作





串行通信

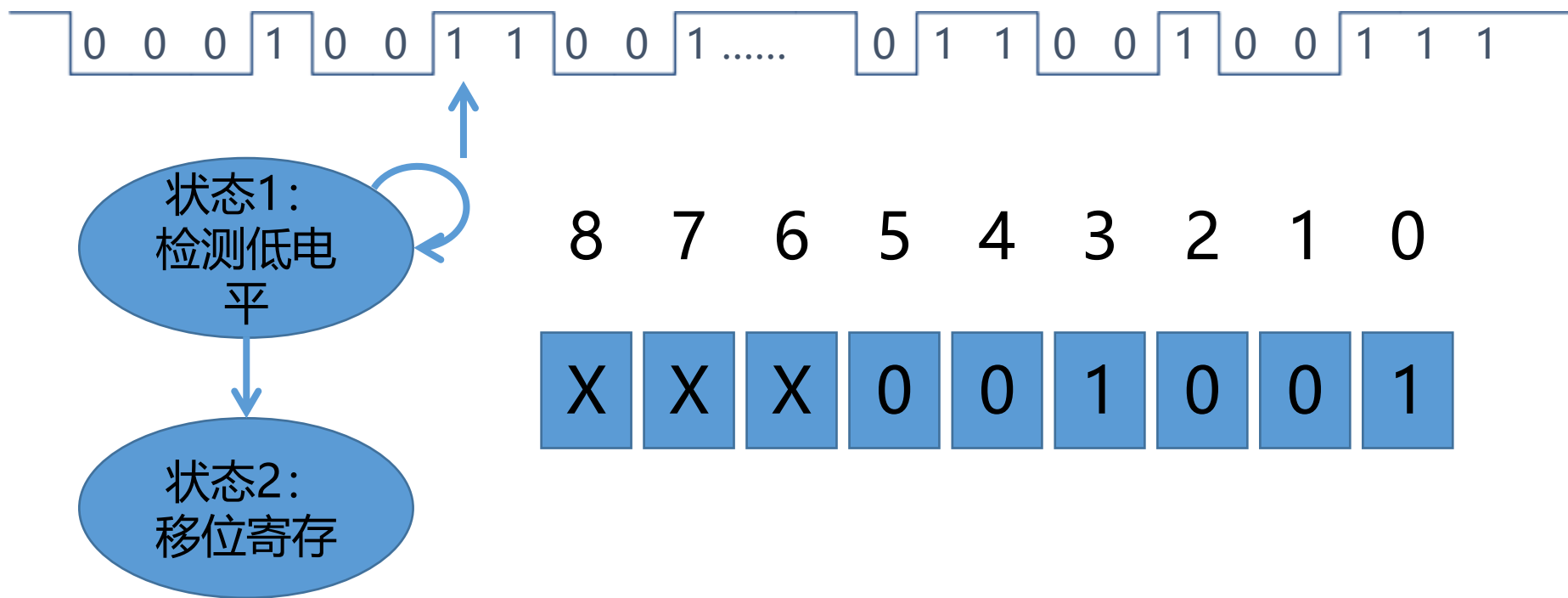
■ 接收端如何工作





串行通信

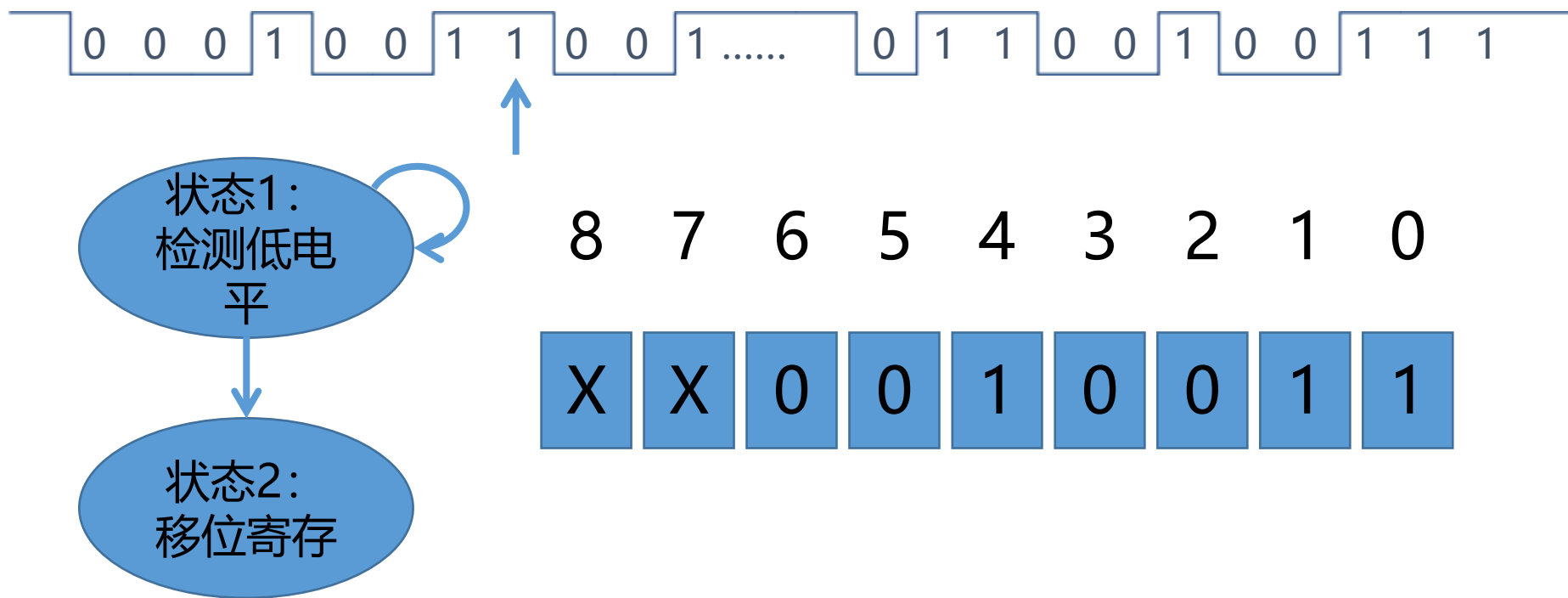
■ 接收端如何工作





串行通信

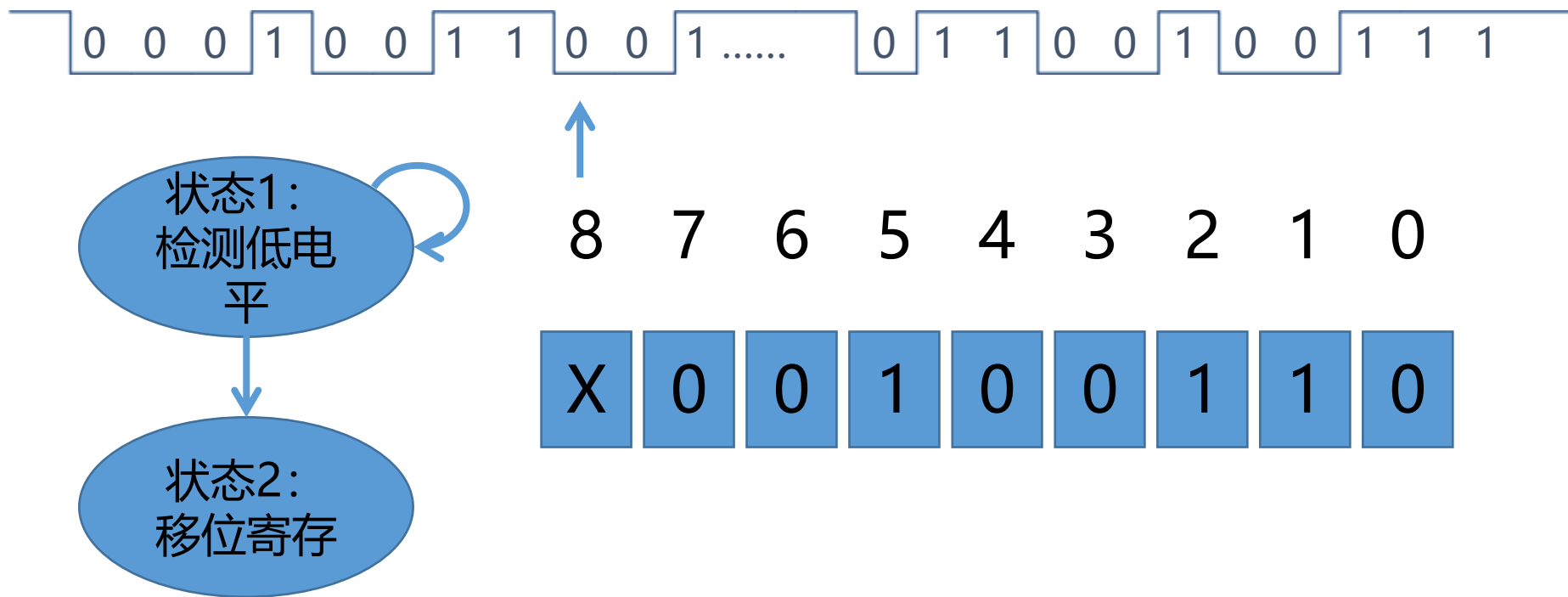
■ 接收端如何工作





串行通信

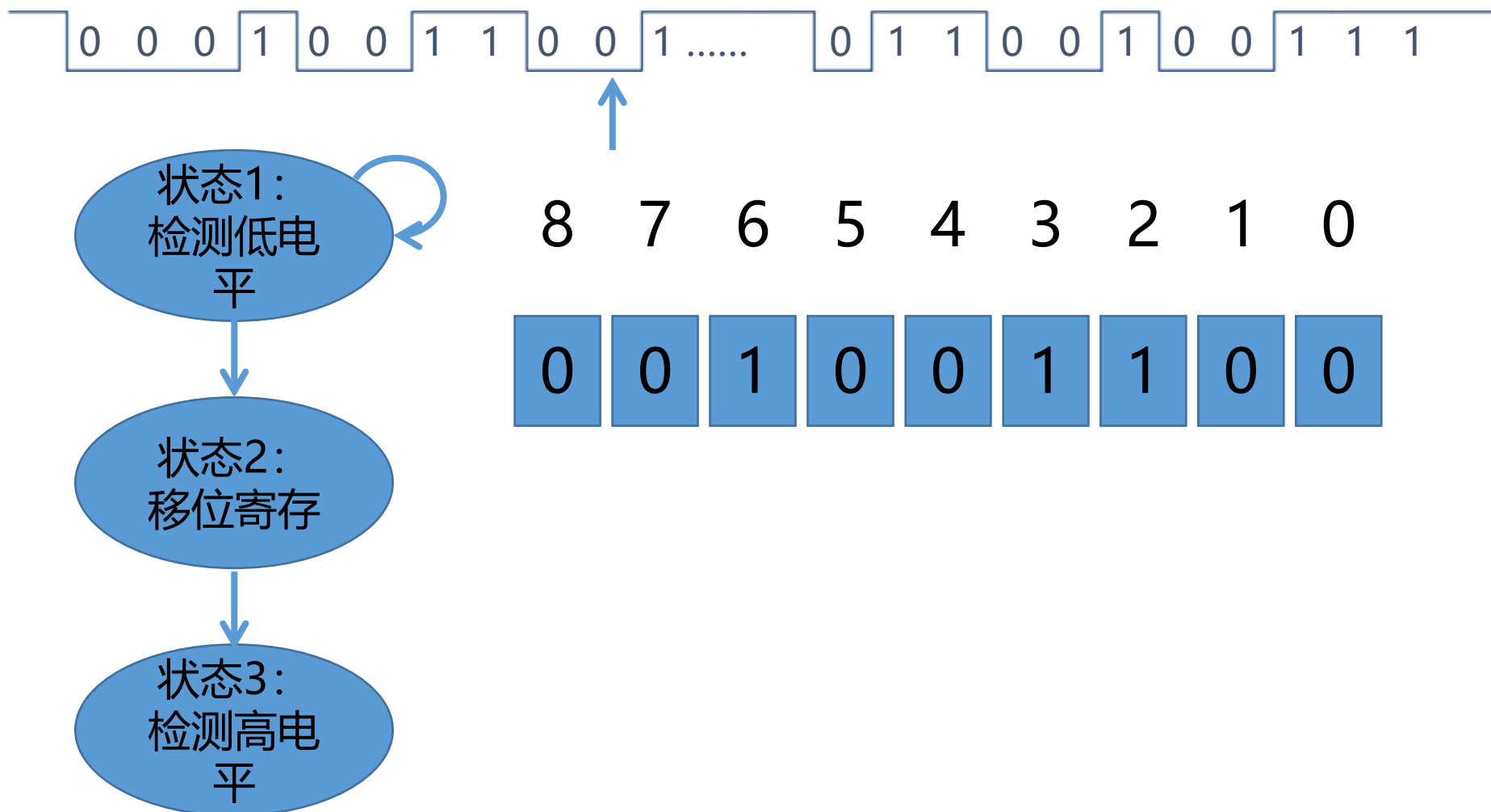
■ 接收端如何工作





串行通信

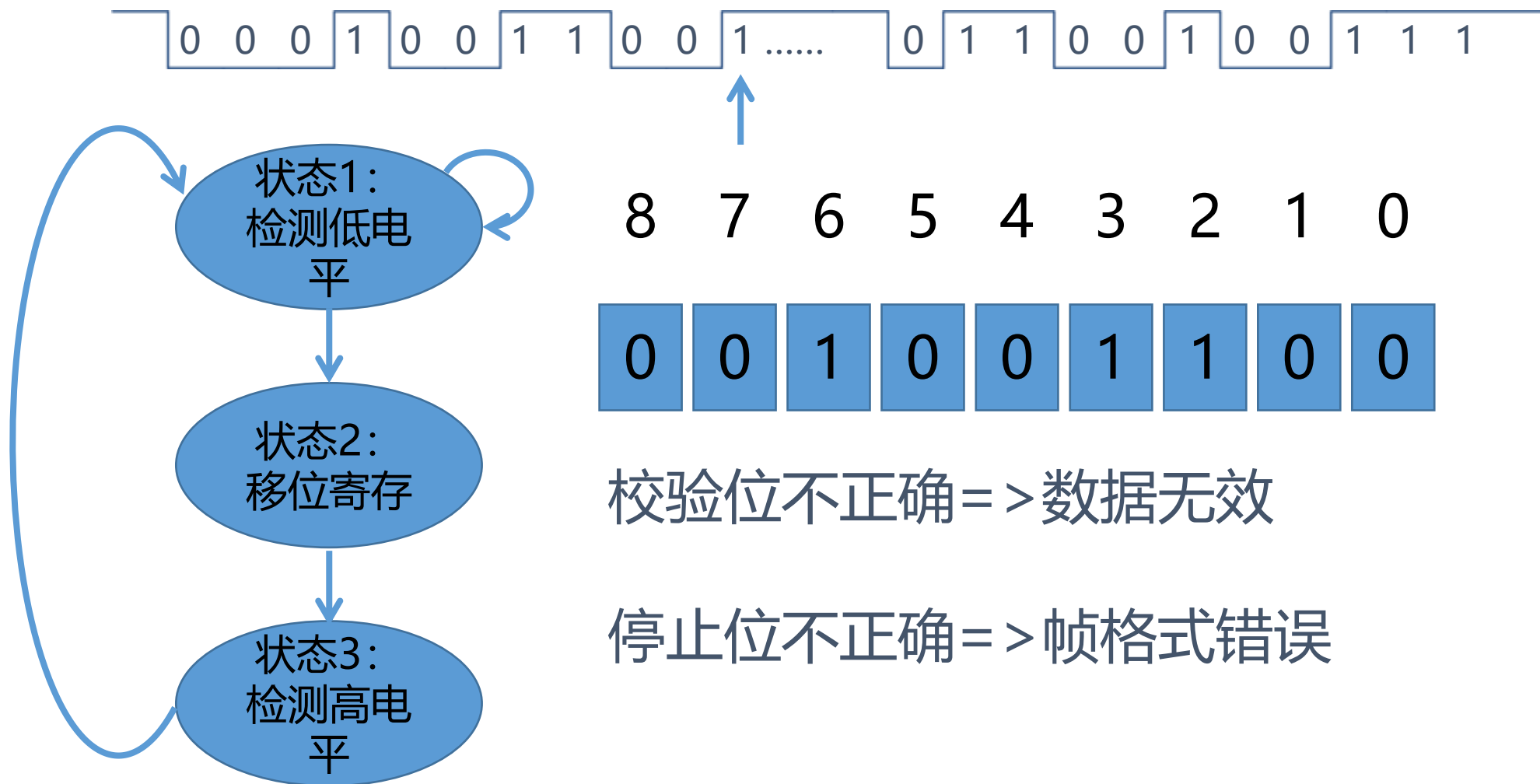
■ 接收端如何工作





串行通信

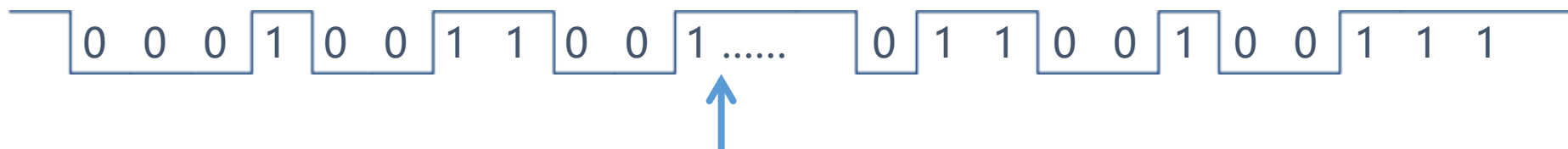
■ 接收端如何工作





串行通信

■ 接收端如何工作



状态1:
检测低电
平

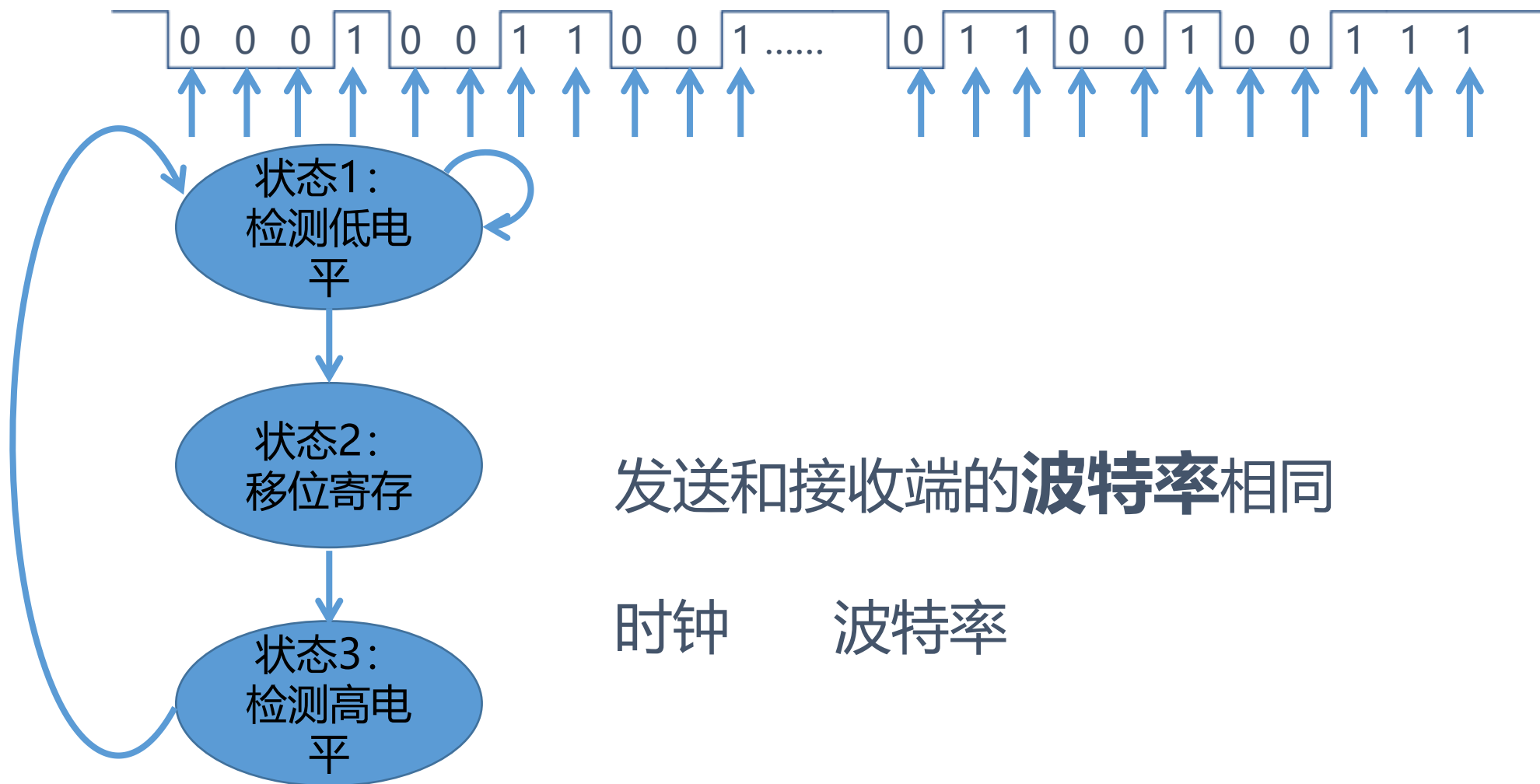
状态2:
移位寄存

状态3:
检测高电
平



串行通信

■ 接收端如何工作





串行通信

■ 比特率、波特率



比特率指每秒传输的二进制数个数，单位为比特/秒
bit/s(bps)

波特率指每秒传输的符号个数，单位为波特(Baud)

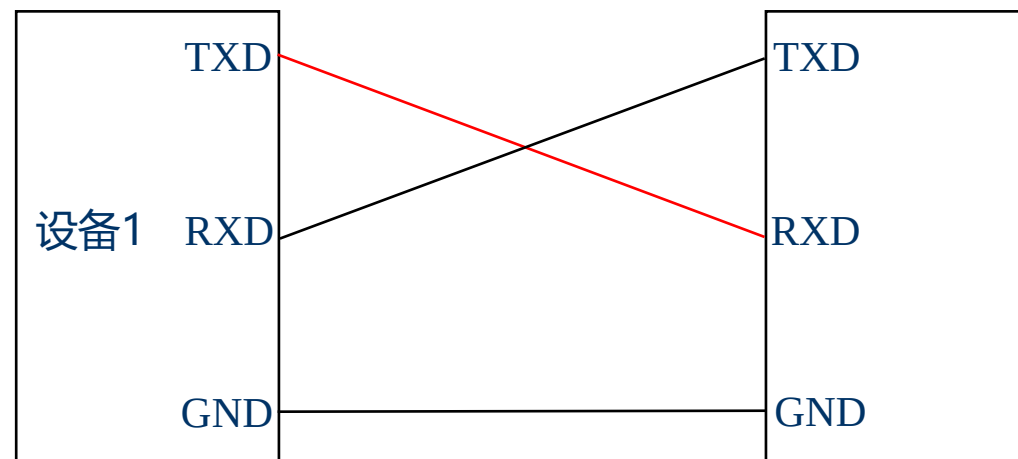
比特率 = 波特率 x 每个符号包含的二进制数个数



串口通信

■ 串口通信概念介绍

- **串口通信**（Serial Communication）是指串口按**位（bit）**发送和接收数据。
- **串口通信**使用3根线完成，分别是地线（GND）、发送线（TX）、接收线（RX）。
- 由于串口通信是**全双工**的，端口能够在同一根线上发送数据同时在另一根线上接收数据。
- 串口通信最重要的参数是**波特率、数据位、停止位和奇偶校验位**。使用串口通信进行通信的两个设备这些参数必须完全**相同**。

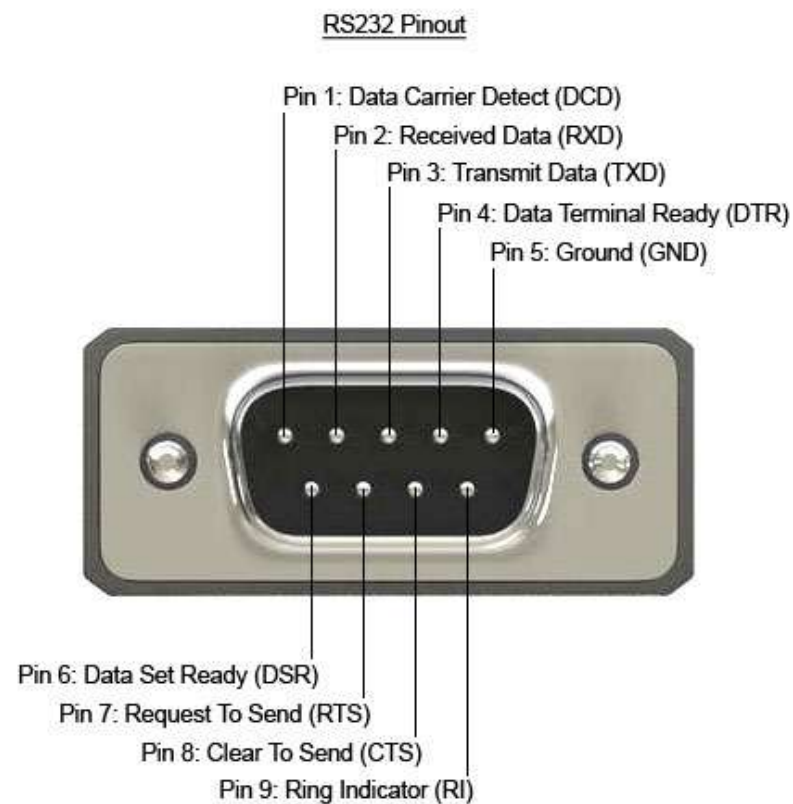




RS232协议

■ 异步模式—RS232

- 电子工业协会 (Electronic Industries Association, EIA) 制定的异步传输标准接口
- 通常 RS-232接口有9个引脚 (DB-9)





RS232协议

■ 异步模式—RS232

在数据线TXD和RXD上:

逻辑1 = - 3V ~ - 15V

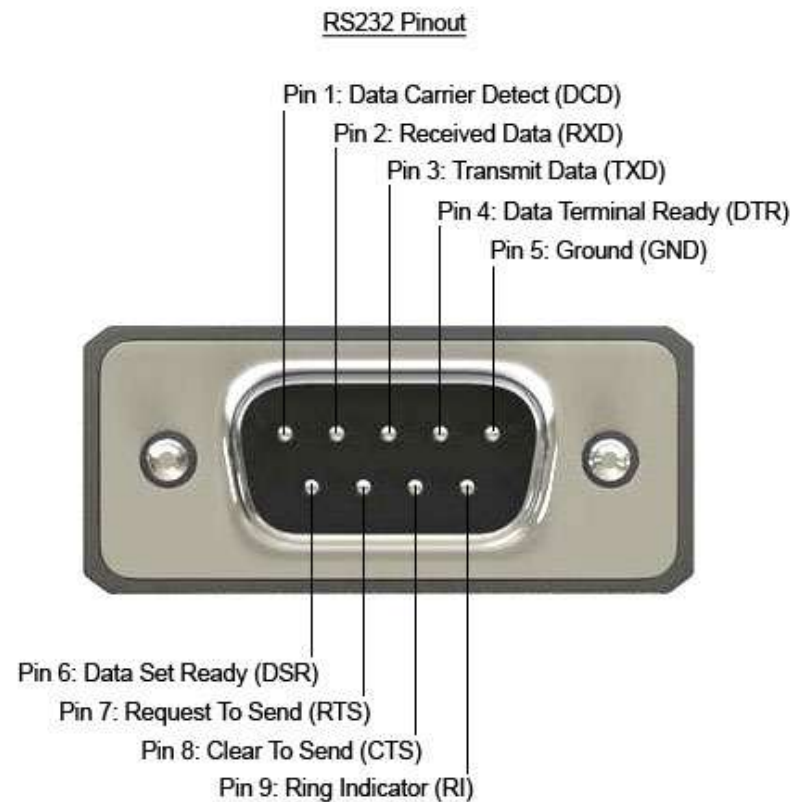
逻辑0 = + 3V ~ + 15V

在控制线和状态线RTS、CTS、DSR、DTR和DCD上:

信号有效 = + 3V ~ + 15V

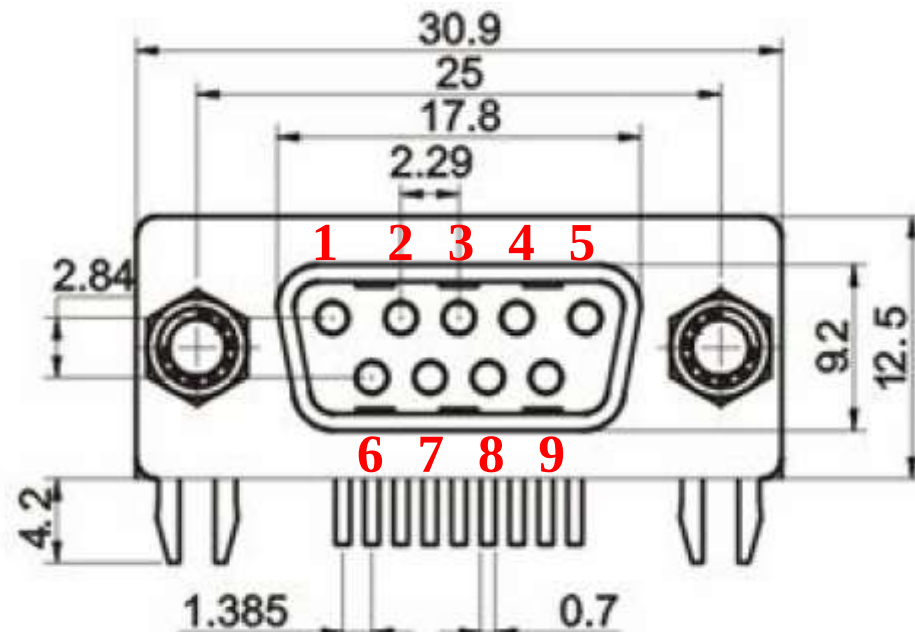
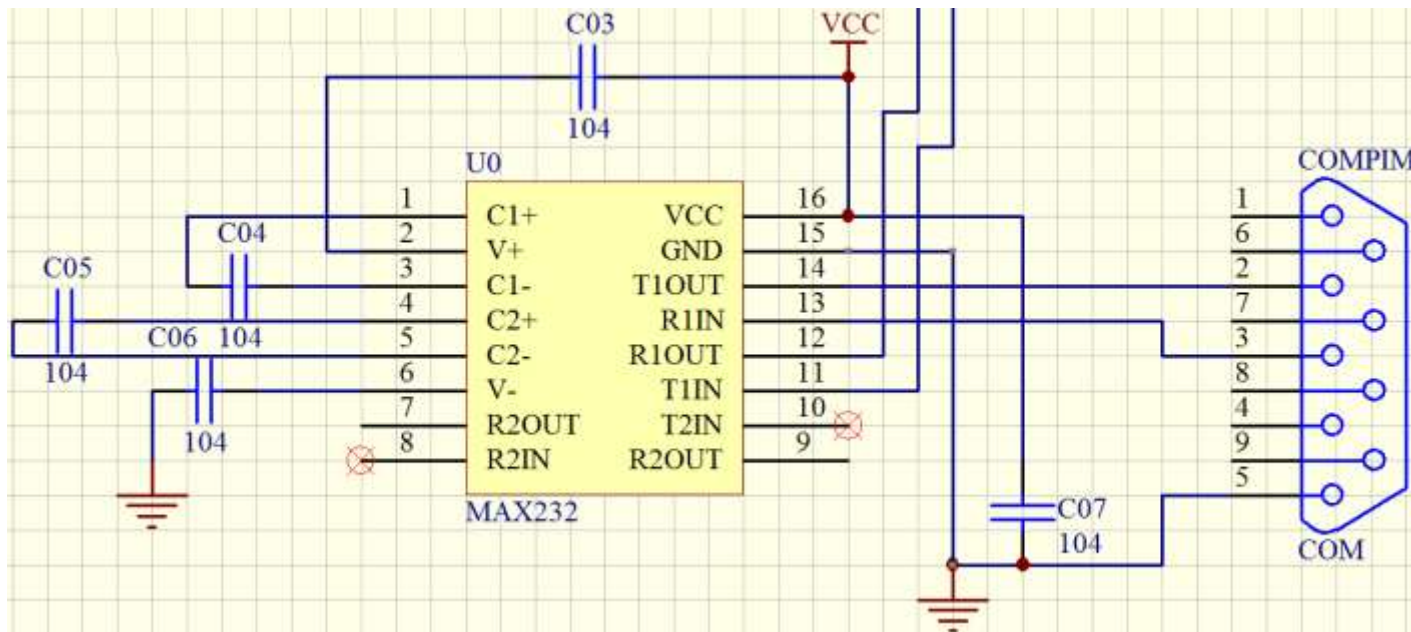
信号无效 = - 3V ~ - 15V

RS - 232C是用负电压来表示逻辑状态,与TTL以高电平表示逻辑状态的规定不同。因此,为了能够使计算机接口与终端的TTL器件连接, **必须在RS - 232与TTL电路之间进行电平和逻辑关系变换。**



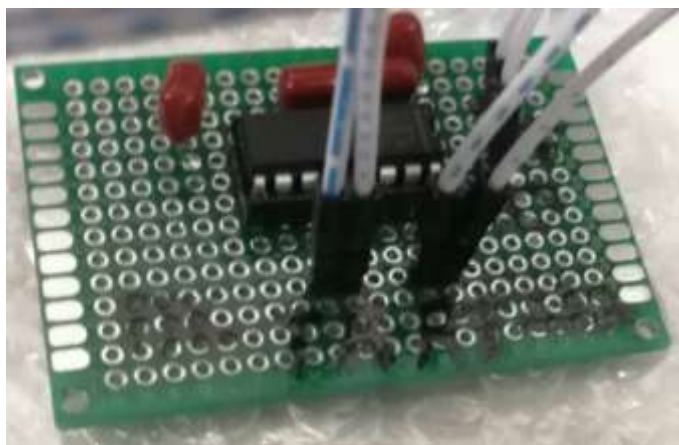


RS-232串行协议（串口通信协议）



MAX232芯片

将单片机输出的TTL电平转换成PC机能接收的232电平，或将PC机输出的232电平转换成单片机能接收的TTL电平。



9针（孔）串口接头

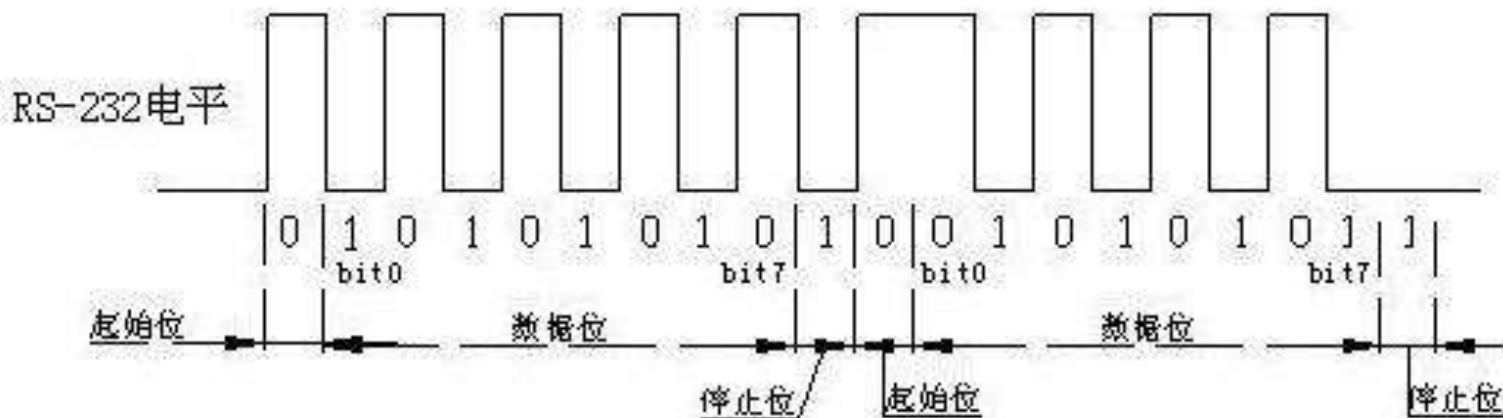
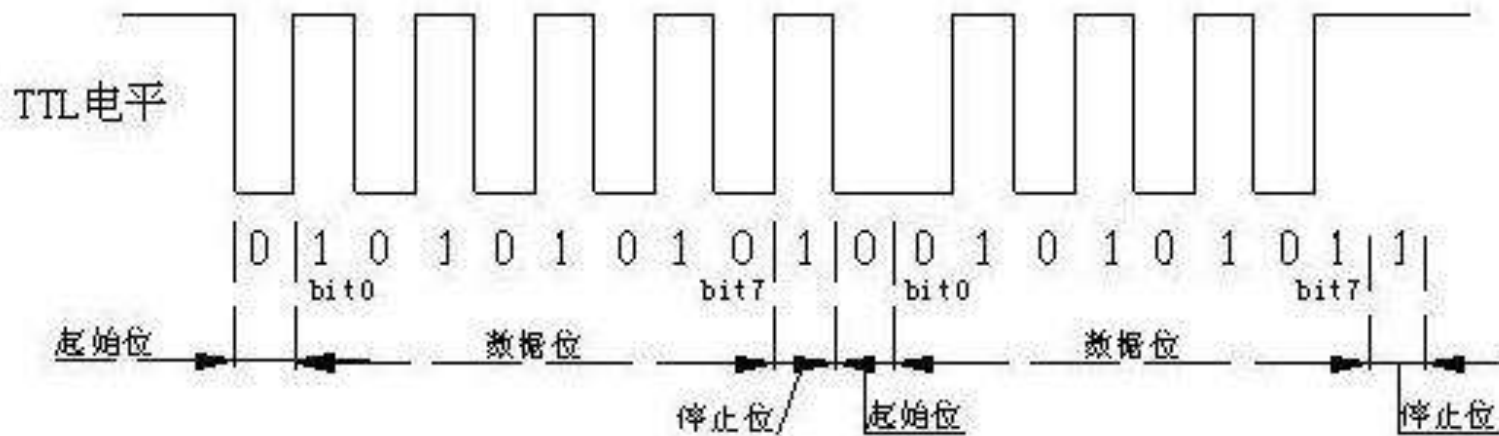
- 2针→TX
- 3针→RX
- 5针→GND



RS-232串行协议（串口通信协议）物理实现

■ 线路空闲时，线路的TTL电平总是高，经反向RS232的电平总是低。

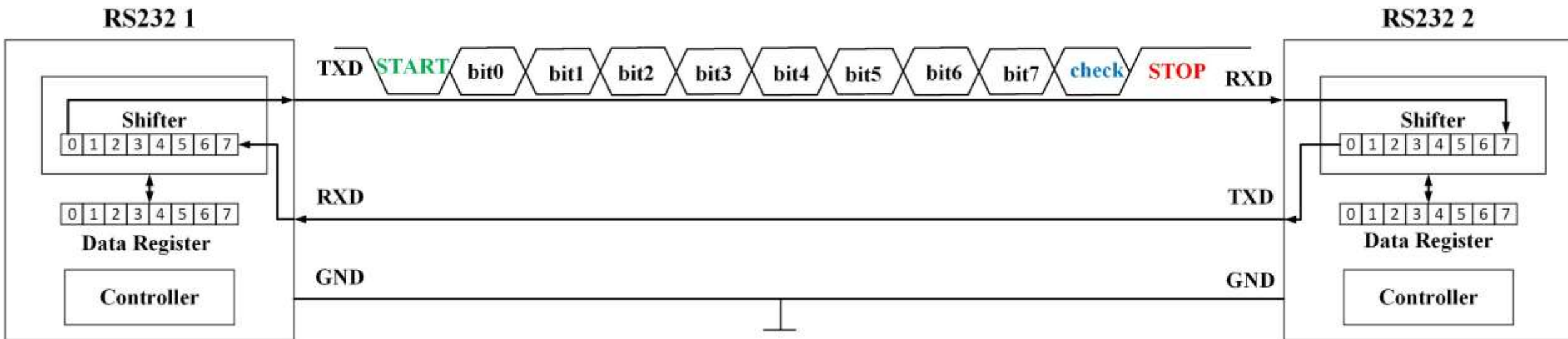
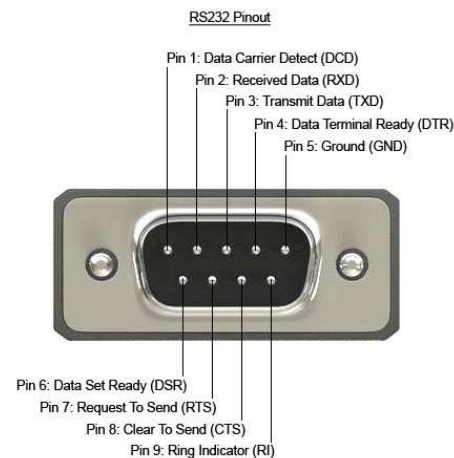
■ 一个数据的**开始**RS232线路为**高电平**，**结束**RS232为**低电平**。





RS232协议

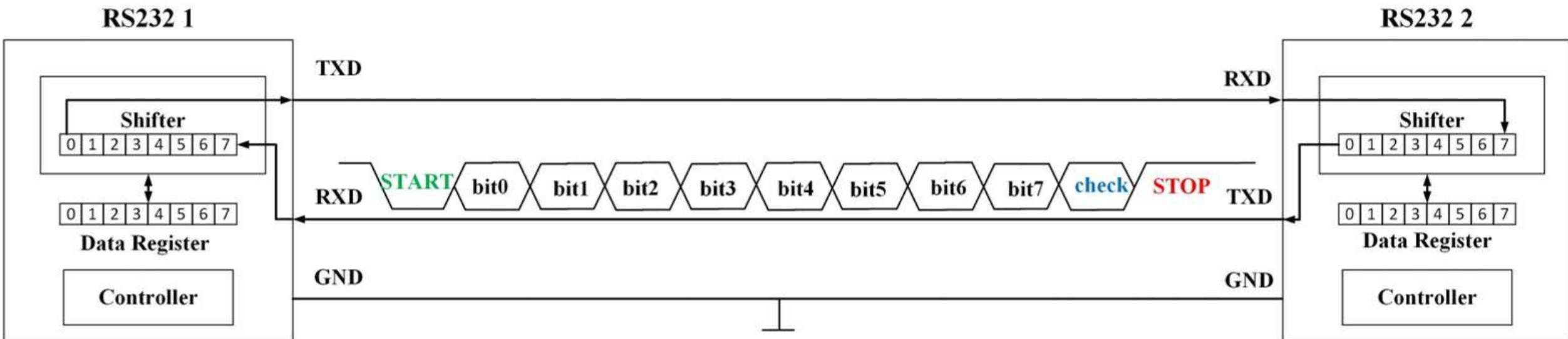
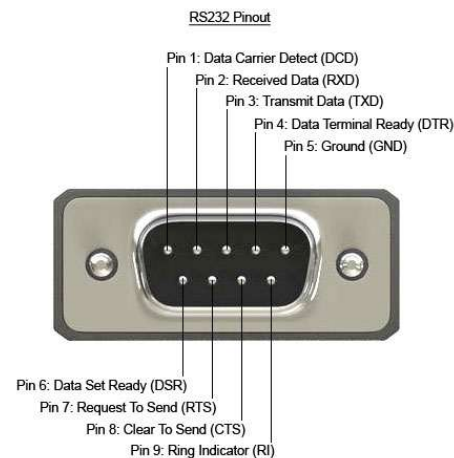
■ 3线连接示意图





RS232协议

■ 3线连接示意图





串口通信函数

- `Serial.begin()`: 开始串口(串口配置);
- `Serial.end()`: 禁止串口传输;
- `Serial.available()`: 判断串口缓冲区的状态;
- `Serial.read()`: 读取串口数据;
- `Serial.print()`: 串口输出数据;
- `Serial.println()`: 串口输出数据并换行;
- `Serial.parseInt()`: 从数据流中读取第一个有效整数;



串口通信函数

■ Serial.begin() :

- 该函数用于设置串口的波特率，通常置于setup()函数中。无返回值。
- 语法：Serial.begin(speed); Serial.begin(speed,config);
- 参数speed：设置串口波特率；参数config：设置数据位、奇偶校验和停止位，默认值为8个数据位，**无奇偶校验位**，1个停止位

■ Serial.end() :

- 禁用串口，此时引脚20和21可以用于输入输出，无传递参数，无返回值
- 语法：Serial.end()



串口通信函数

■ Serial.available():

- 该函数用来**判断串口是否收到数据**，函数的返回值为从串口缓冲区读取的字节数，为int型，无传递参数；串口缓冲区最多可以缓存64个字节的数据。
- 语法：Serial.available()

■ Serial.read():

- **将串口数据读入**，一次读取一个字节，读完后从缓存中删除已读取数据。该函数不带参数，返回值为串口缓存区中第一个可读字节，int型，当没有可读取数据时返回-1。
- 语法：Serial.read()

(如果要把读取到的数据转化为字符类型需要怎么做？)

强制类型转换：char(Serial.read())



串口通信函数

■ Serial.print():

- 串口输出数据，将数据作为**可读ASCII文本**输出到串口，返回值为**写入数据的字节数**，该返回值可选择是否读。
- 语法：Serial.print(val); Serial.print(val, format)
- 参数val：需要输出的值，支持任意数据格式；参数format:输出的数据格式，包括BIN(二进制)OCT(八进制)、DEC(十进制)、HEX(十六进制)等对于浮点数，此参数指定要使用的小数位数

■ Serial.println():

- 该函数与Serial.print（）类似，只是多了**换行功能**



串口通信函数

■ Serial.parseInt():

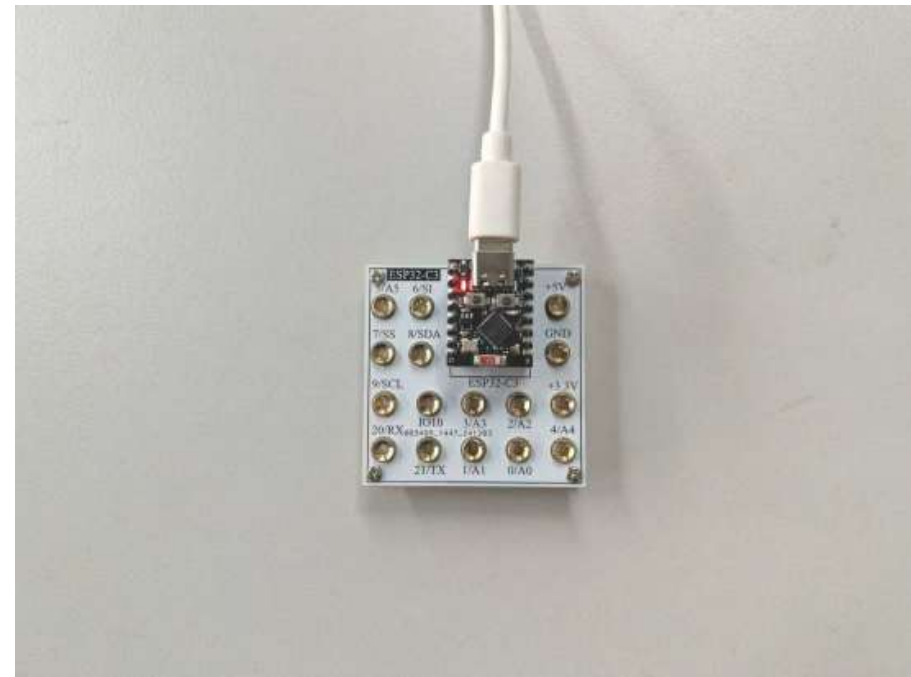
- 从串口接收数据流中读取**第一个有效整数**（该函数会跳过不是数字或负号的初始字符，当可配置的超时值读不到有效字符或者有效整数时，分析停止）；**返回值为下一个有效整数值**
- 语法：Serial.parseInt(); Serial.parseInt(charskipChar)
- 参数charskipchar：在搜索中跳过指定字符



实验1：电脑与开发板通信，发送字符

■ 实验内容

- 电脑作为主设备，ESP32开发板为从设备
- 从串口监视器进行**字符输入**，ESP32接收并回复相同字符
- Serial.begin() 设置Arduino波特率
- 串口监视器设置电脑波特率
- 注意：**两个波特率应该设置为相同**

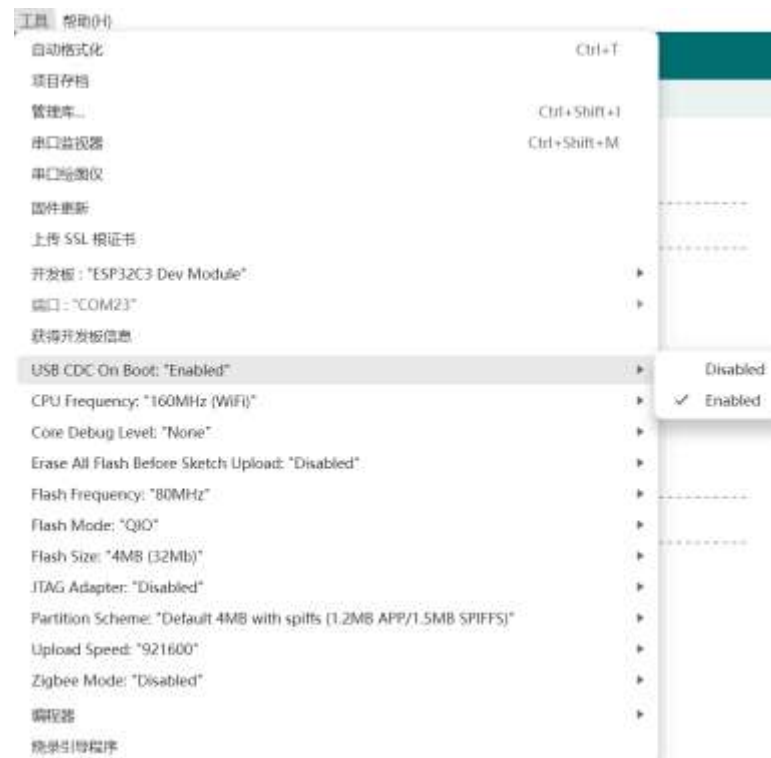
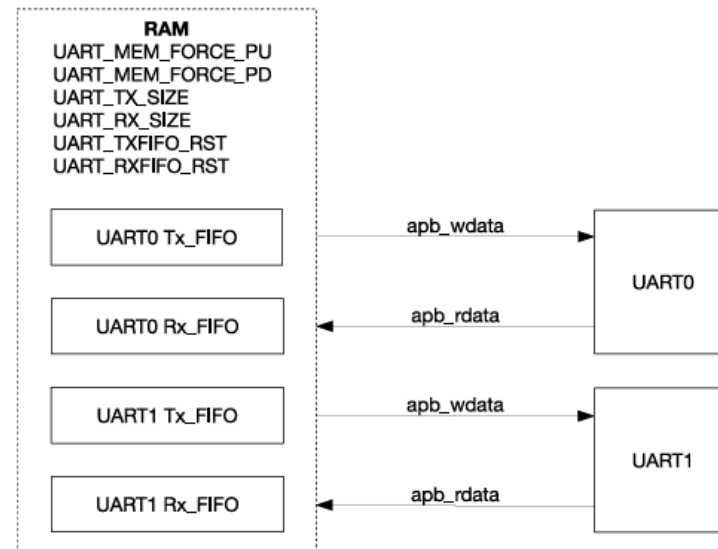




硬件串口

■ ESP32C3的硬件串口

- ESP32C3芯片内部包含了两个硬件串口
- 在Arduino中可以通过Serial和Serial1分别控制ESP32C3的串口0和串口1
- 由于电脑端无法直接和单片机串口通信，需要使用ESP32C3内置的USB CDC功能，在使用USB CDC后，可以使用Serial控制串口0和电脑端通信，此时串口0不能通过数字管脚与外界通信





实验1：电脑与开发板通信，发送字符

■ 实验代码

uart_char.ino

```
1  char x = 0;
2  void setup() {
3      // put your setup code here, to run once:
4      Serial.begin(9600); //初始化串口，设置波特率为9600
5  }
6
7  void loop() {
8      // put your main code here, to run repeatedly:
9      if(Serial.available() > 0)
10     {
11         x = Serial.read();
12         Serial.print("I have received:");
13         Serial.println(x); //输出并换行
14     }
15     delay(200);
16 }
17
```



实验2：示波器观察串口波形

■ 实验内容

- 使用示波器观察开发板串口的TX发送引脚的波形
- 学习使用示波器查看波形的基本方法
- 修改Serial1.begin中波特率和帧格式参数，观察不同参数下的波形
- 截图并标示码元宽度，校验位，测量帧周期

```
Serial.begin(9600,SERIAL_8N1);
```

```
Serial.begin(9600,SERIAL_8N1)
```

第一个数字代表每一帧包含的数据位数，字母代表有无校验码（N：无校验，O：奇校验，E：偶校验），第二个数字代表停止位的位数



实验2：示波器观察串口波形

■ 实验代码

```
#define UART1_RX_Pin 20
#define UART1_TX_Pin 21

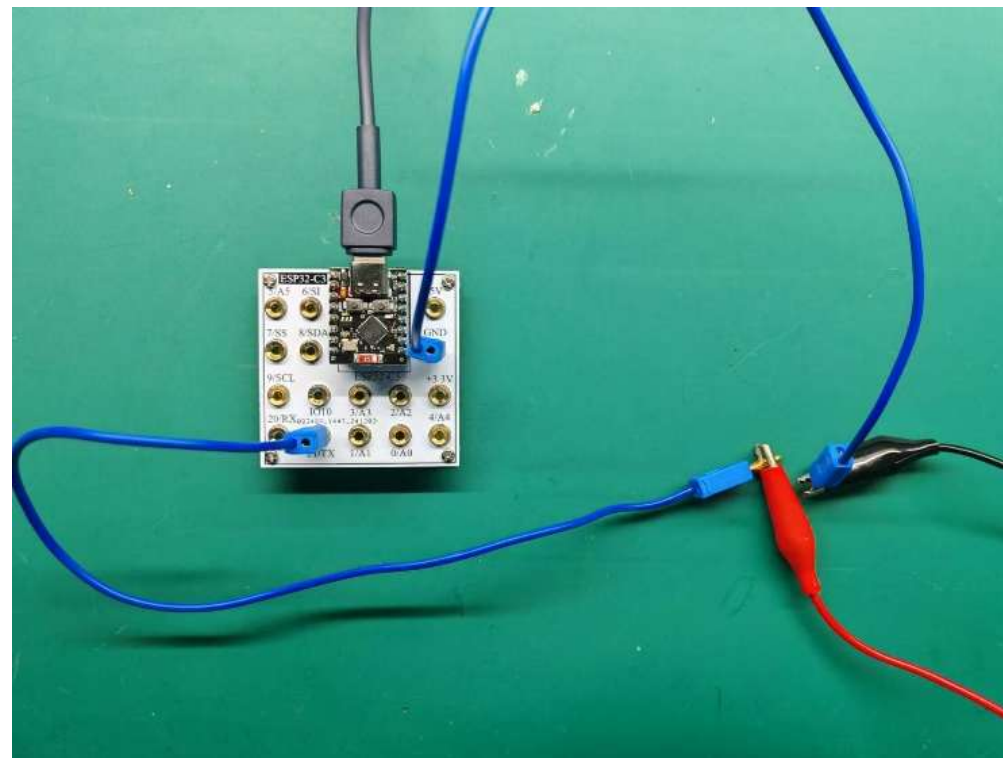
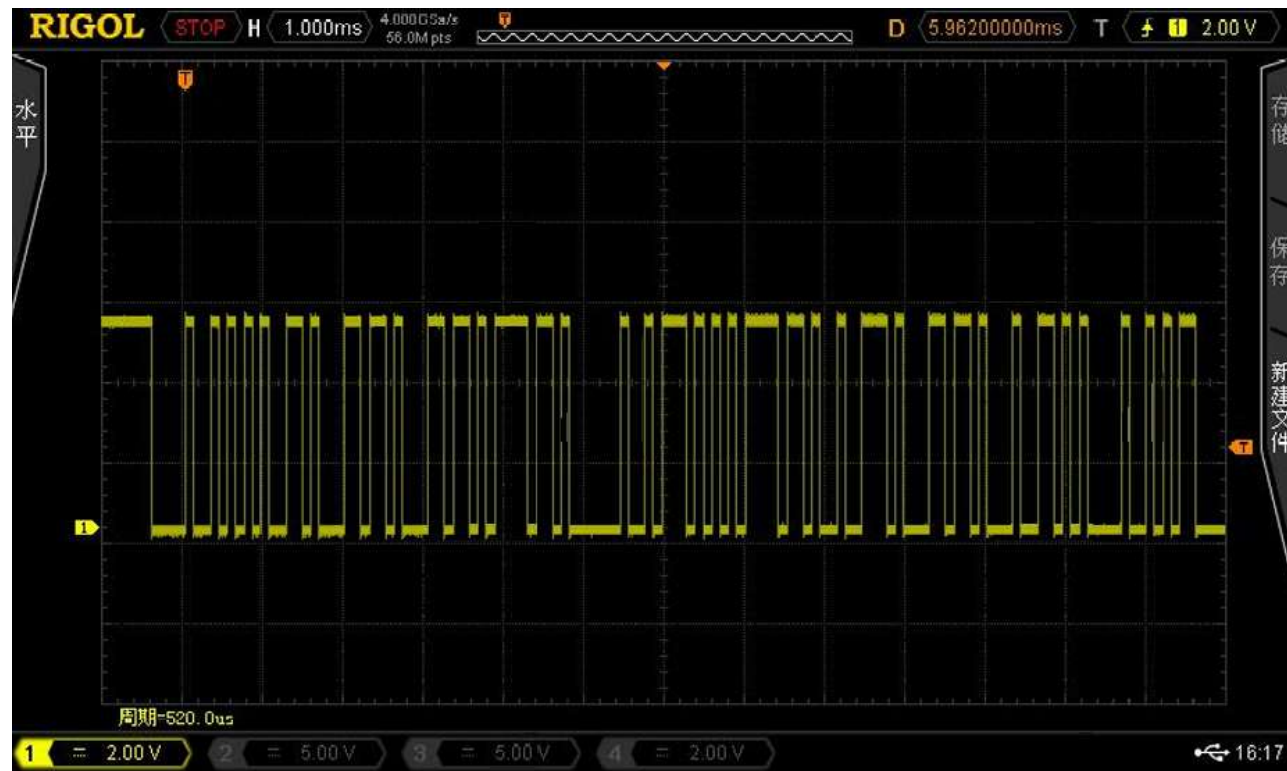
void setup() {
    //使用硬件串口1，将Pin20作为RX引脚，Pin21作为Tx引脚
    Serial1.begin(9600,SERIAL_8N1,UART1_RX_Pin,UART1_TX_Pin);
}

void loop() {
    Serial1.println("Hello World!");
    delay(200);
}
```

二进制	十进制	十六进制	图形	二进制	十进制	十六进制	图形
0100 0000	64	40	@	0110 0000	96	60	`
0100 0001	65	41	A	0110 0001	97	61	a
0100 0010	66	42	B	0110 0010	98	62	b
0100 0011	67	43	C	0110 0011	99	63	c
0100 0100	68	44	D	0110 0100	100	64	d
0100 0101	69	45	E	0110 0101	101	65	e
0100 0110	70	46	F	0110 0110	102	66	f
0100 0111	71	47	G	0110 0111	103	67	g
0100 1000	72	48	H	0110 1000	104	68	h
0100 1001	73	49	I	0110 1001	105	69	i
0100 1010	74	4A	J	0110 1010	106	6A	j
0100 1011	75	4B	K	0110 1011	107	6B	k
0100 1100	76	4C	L	0110 1100	108	6C	l
0100 1101	77	4D	M	0110 1101	109	6D	m
0100 1110	78	4E	N	0110 1110	110	6E	n
0100 1111	79	4F	O	0110 1111	111	6F	o
0101 0000	80	50	P	0111 0000	112	70	p
0101 0001	81	51	Q	0111 0001	113	71	q
0101 0010	82	52	R	0111 0010	114	72	r
0101 0011	83	53	S	0111 0011	115	73	s
0101 0100	84	54	T	0111 0100	116	74	t
0101 0101	85	55	U	0111 0101	117	75	u
0101 0110	86	56	V	0111 0110	118	76	v
0101 0111	87	57	W	0111 0111	119	77	w
0101 1000	88	58	X	0111 1000	120	78	x
0101 1001	89	59	Y	0111 1001	121	79	y
0101 1010	90	5A	Z	0111 1010	122	7A	z
0101 1011	91	5B	[	0111 1011	123	7B	{
0101 1100	92	5C	\	0111 1100	124	7C	



实验2：示波器观察串口波形

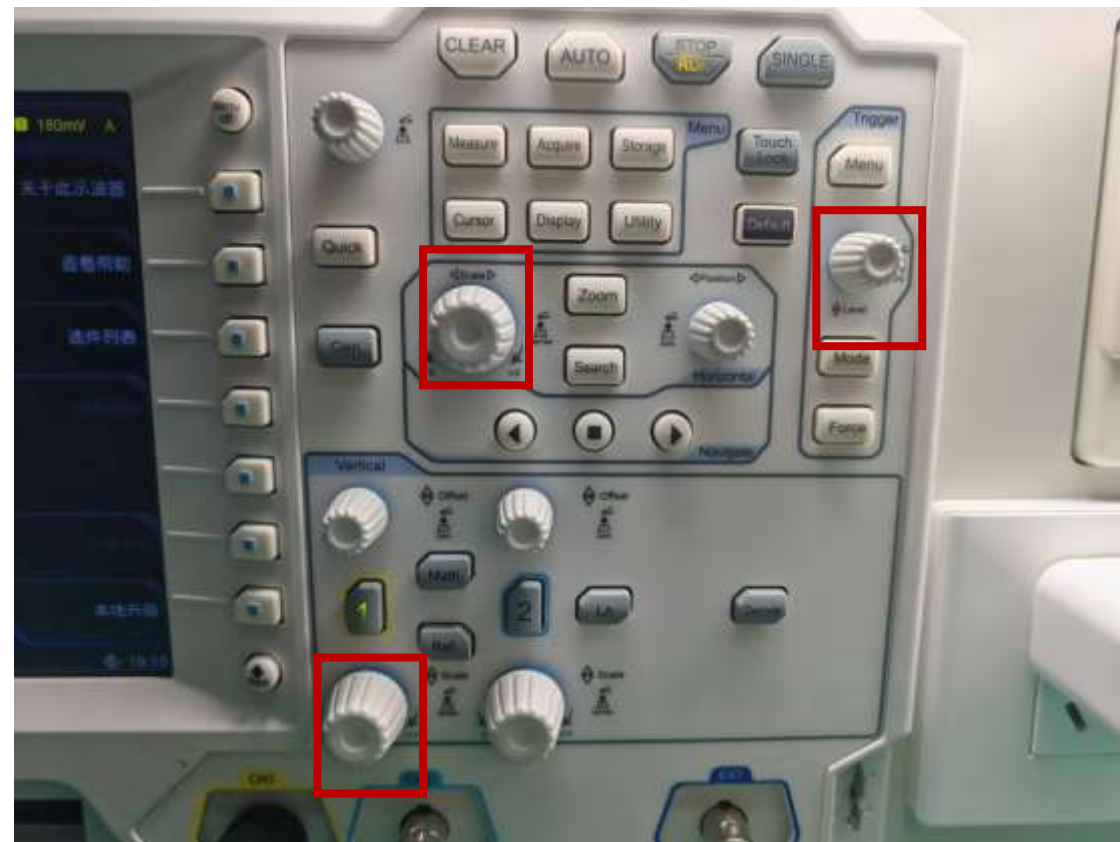




实验2：示波器观察串口波形

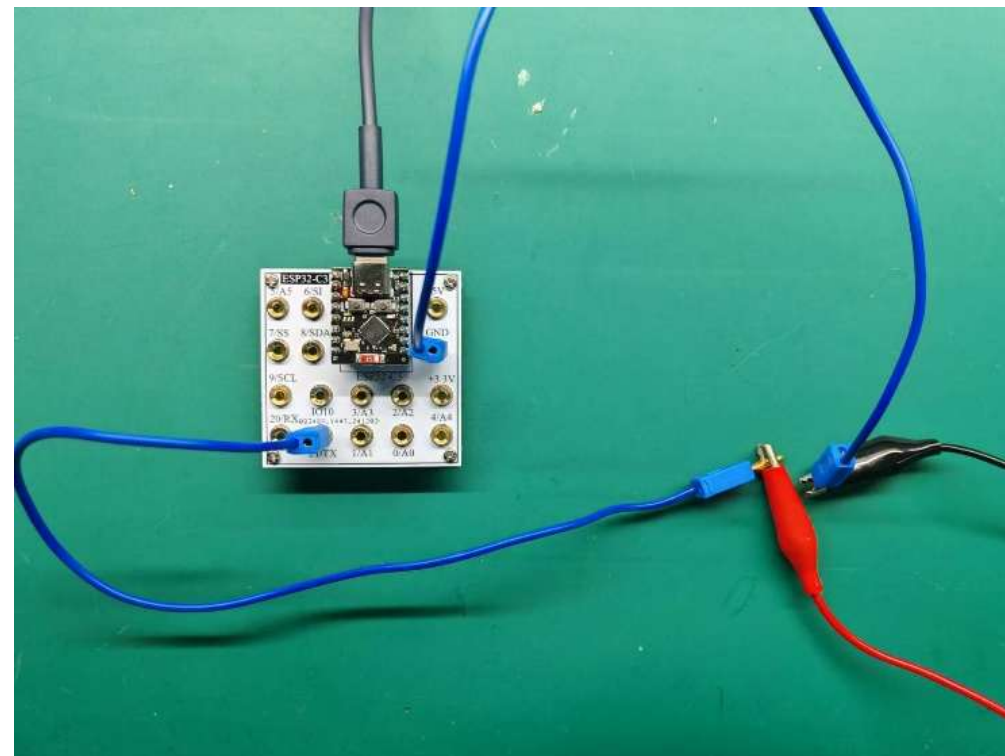
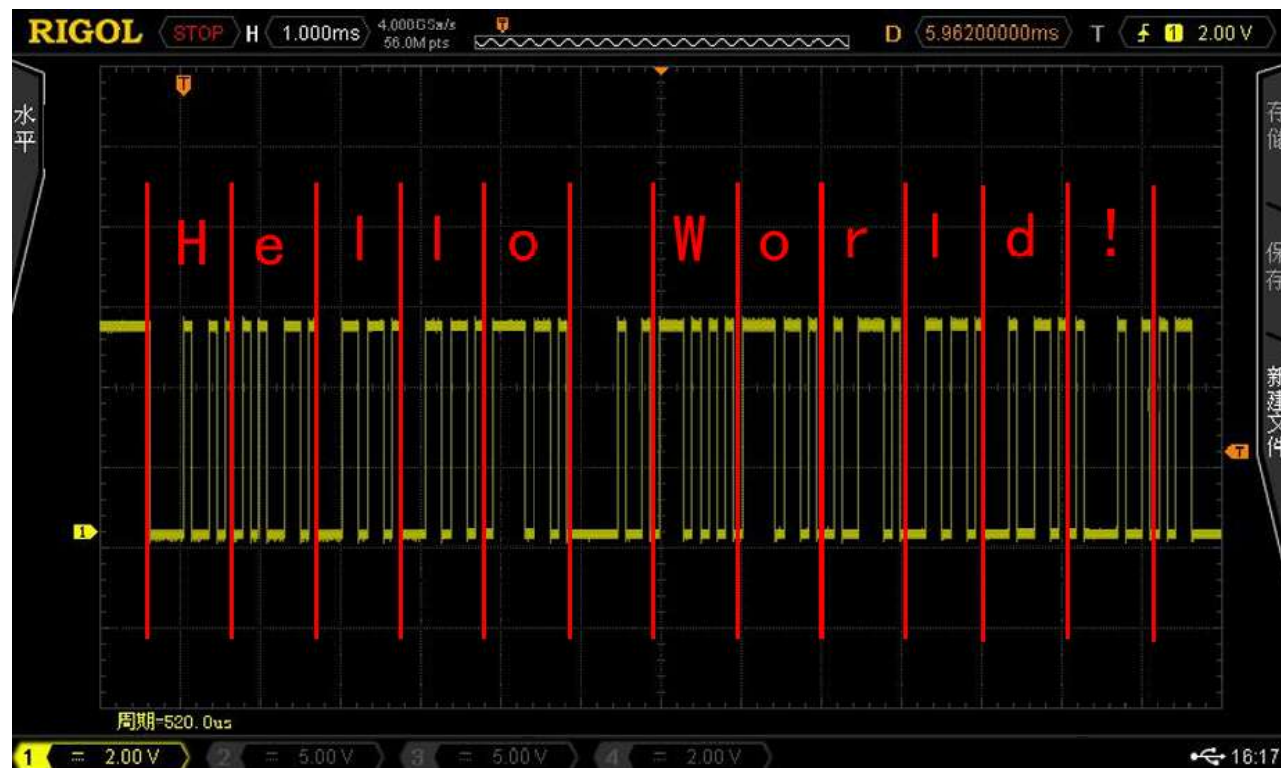
■ 示波器操作步骤

- 转动Scale旋钮直到能够看清波形形状
- 如果发现波形存在抖动，转动Trigger旋钮直到波形不再抖动
- 转动当前通道的Vertical旋钮调整到适合波形的幅值



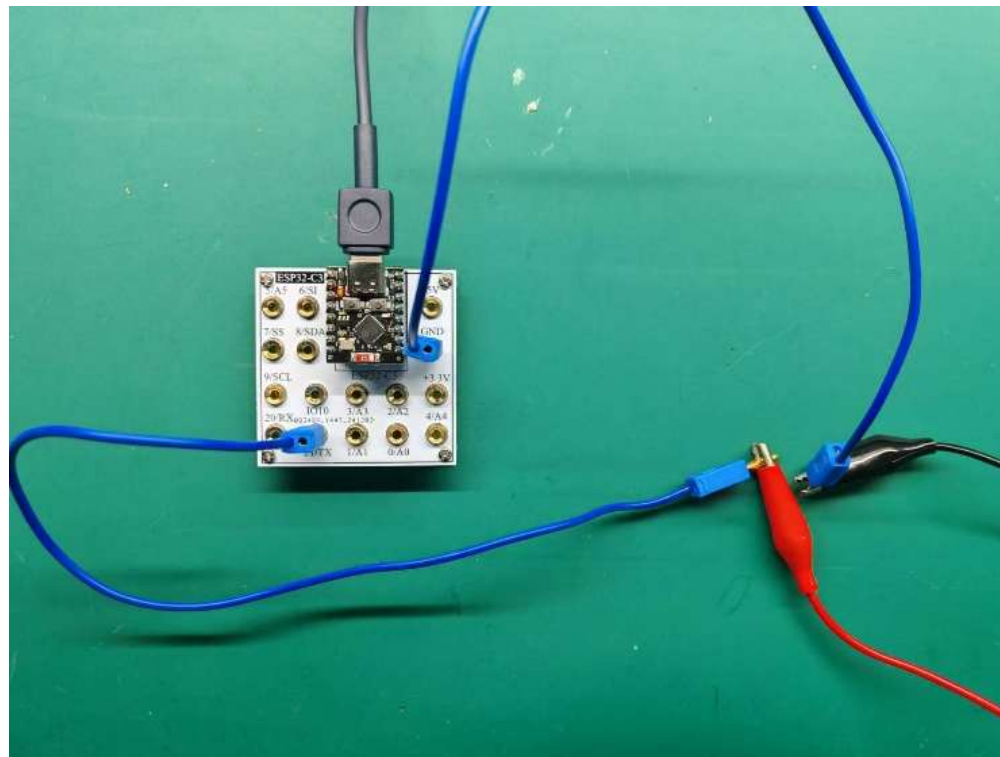
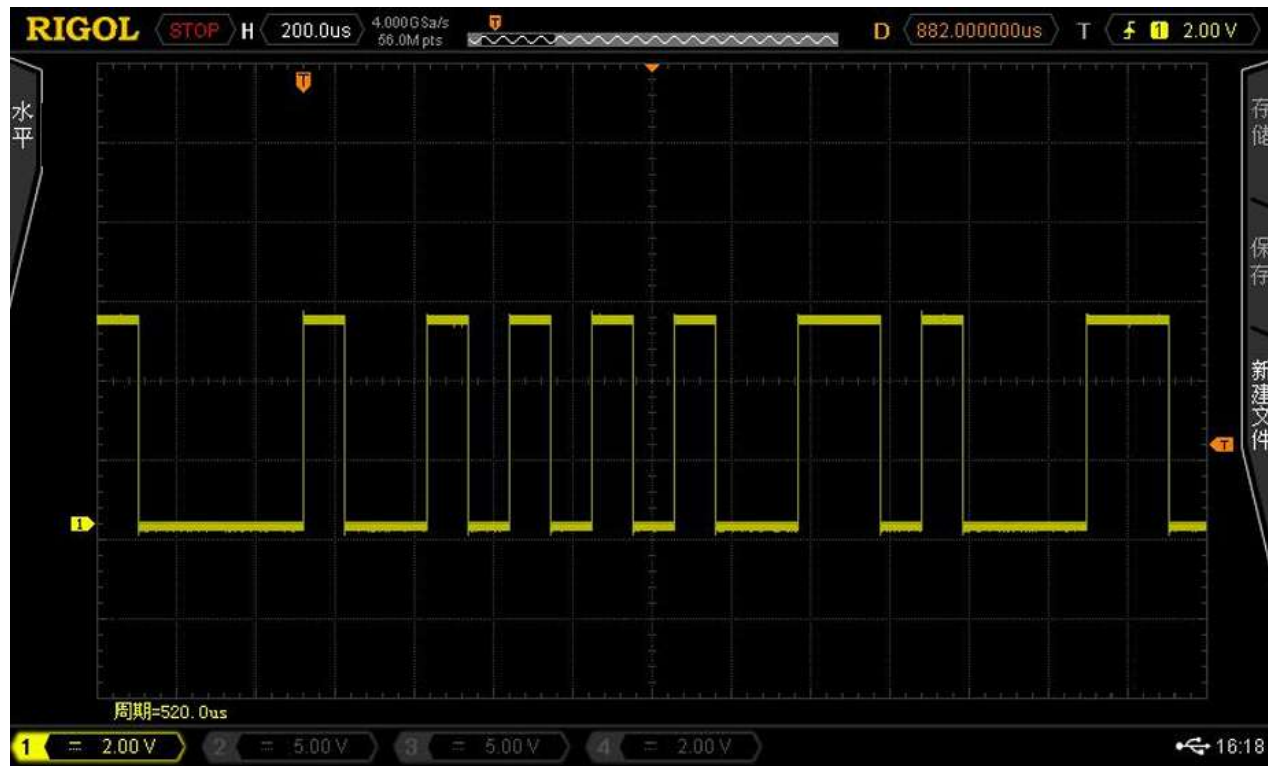


实验2：示波器观察串口波形



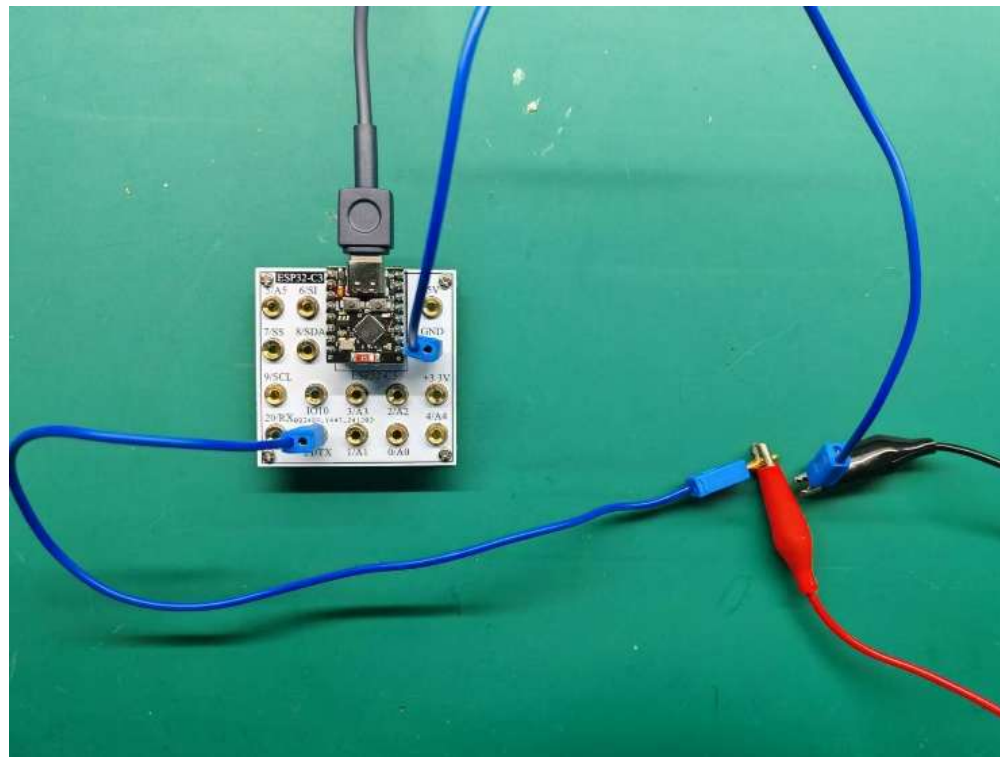
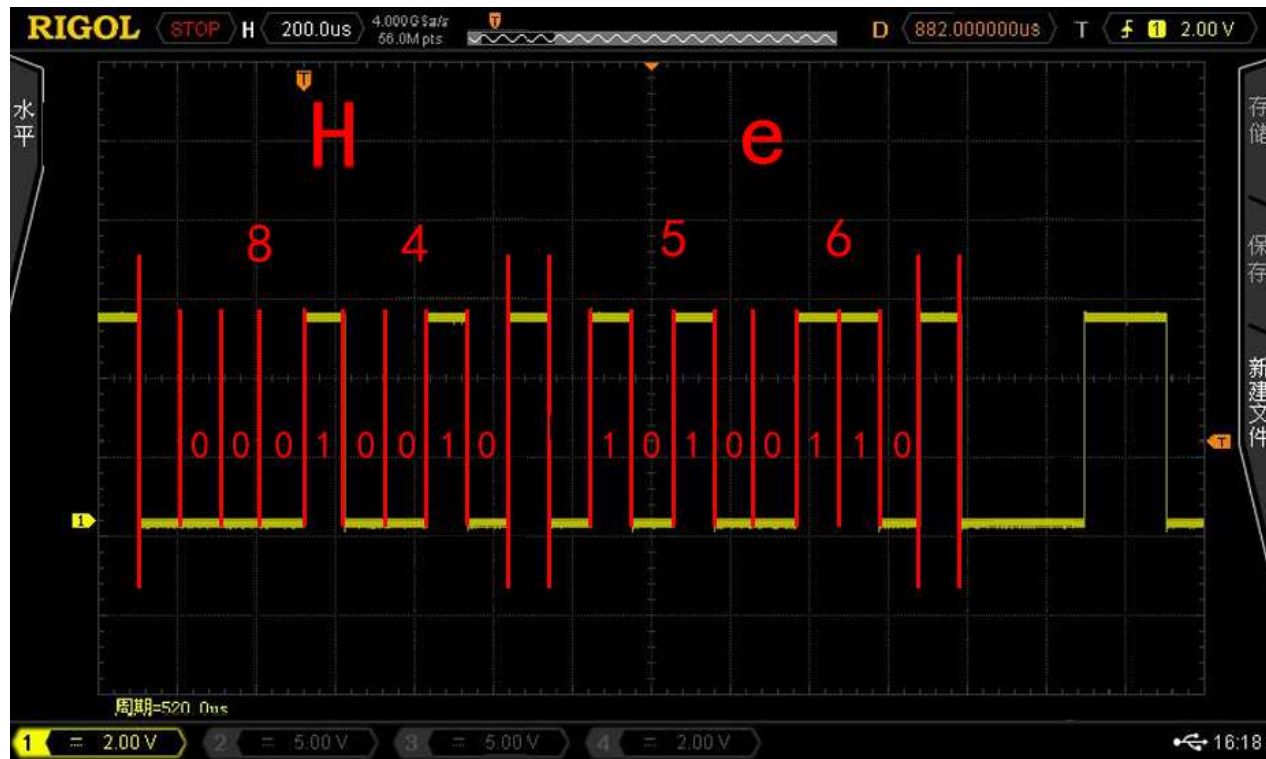


实验2：示波器观察串口波形





实验2：示波器观察串口波形





实验3：电脑与开发板通信，发送字符串

■ C语言知识点复习

- 字符变量（char）：C语言基本数据类型之一，通常占一个字节大小，声明与初始化：char TestChar = 'A';
- 字符串：由字符数组实现并且**以空字符 '\0' 结尾**的字符序列，声明与初始化 char TestString[] = "Hello";

■ Arduino中的String类

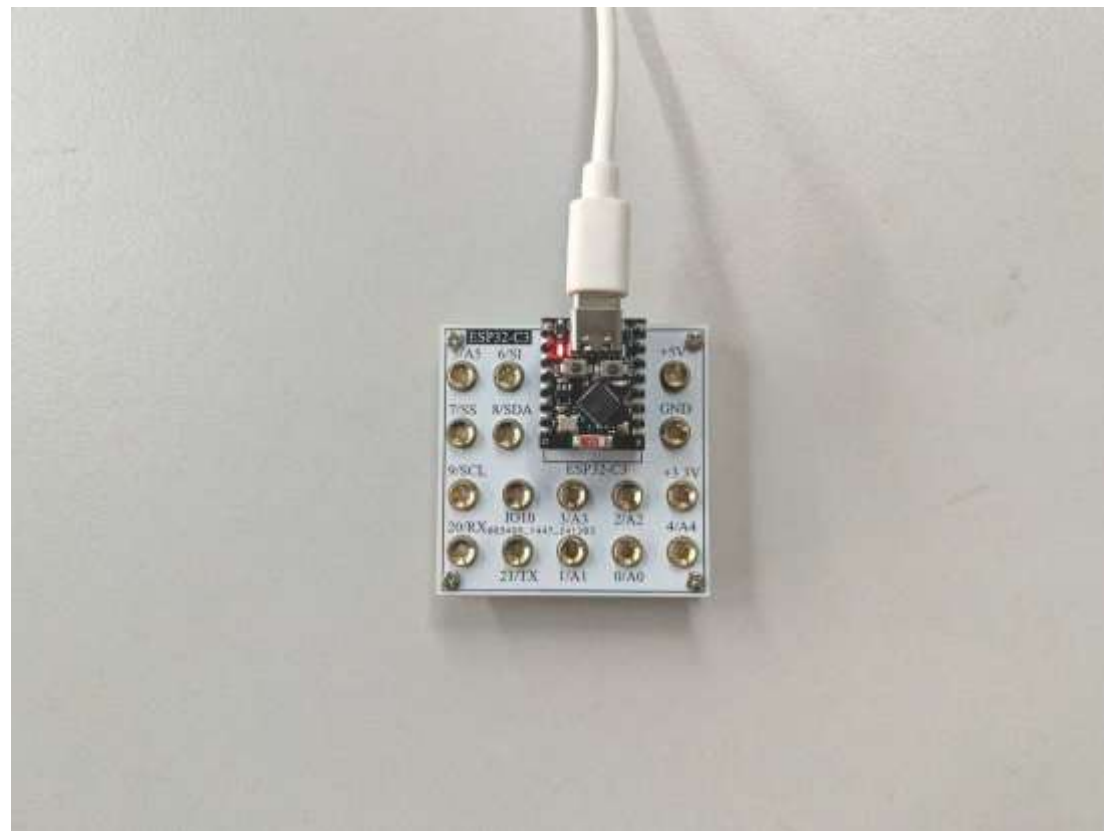
- Arduino为了简化字符串操作提供了String类，String类允许使用+操作符来连接字符串，或者使用+=来追加内容。



实验3：电脑与开发板通信，发送字符串

■ 实验内容

- 在实验1的基础上将串口字符收发修改为字符串收发
- 学习使用Arduino提供的String类简化字符串操作





实验3：电脑与开发板通信，发送字符串

■ 实验代码

uart_test.ino

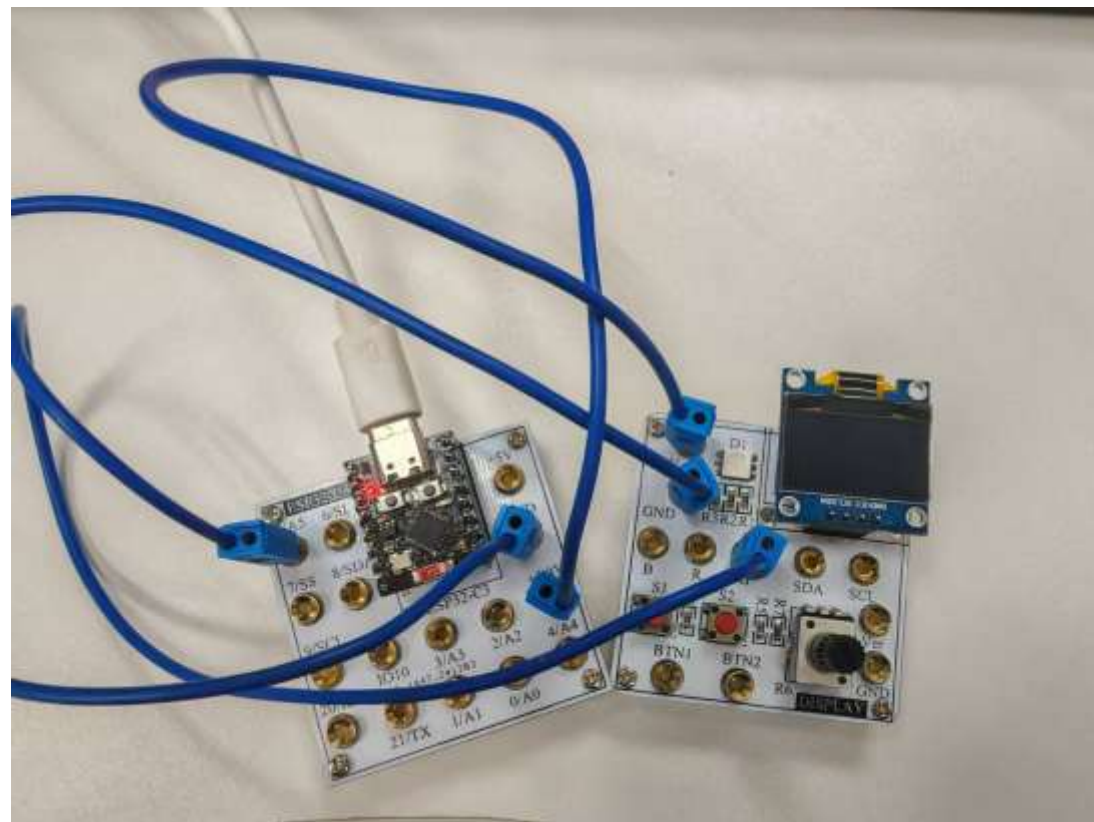
```
1  String inputStr = "";
2  char inputchar = 0;
3
4  void setup() {
5      Serial.begin(9600);
6
7  }
8
9  void loop() {
10     if(Serial.available())
11     {
12         while(Serial.available())
13         {
14             inputchar = Serial.read();//去读接收到的字符
15             inputStr += inputchar;//将接收到的字符拼接到字符串中
16             delay(2);
17         }
18         Serial.println("Received String:"+inputStr);
19         inputStr.remove(0);//清除已接收的字符串
20     }
21 }
22
23
24
```



实验4：控制开发板的IO口

■ 实验内容

- 结合前几章学习到的知识实现串口指令控制单片机IO口
- 具体要求如下：
 - 计算机串口监视器输入1-9任意一个数字（字符形式），5引脚连接的LED闪烁输入数字的次数（闪烁周期为1秒）





实验4：控制开发板的I/O口

■ 实验代码

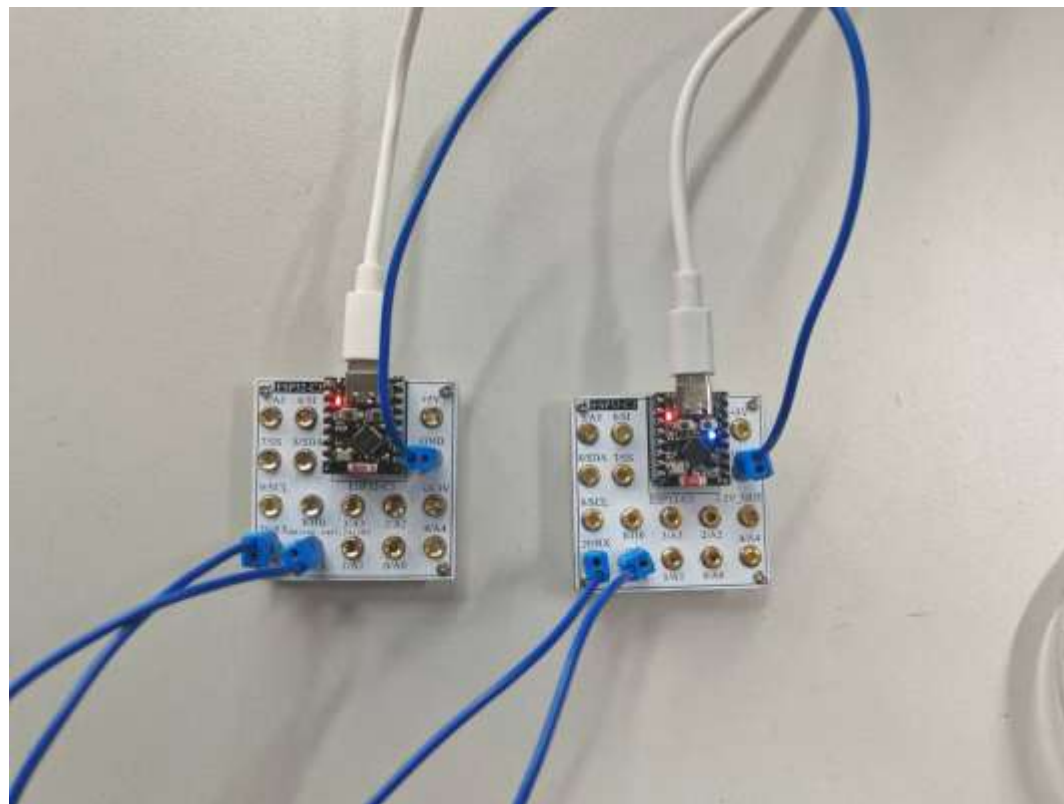
```
1 char receivedChar = 0;
2 int blinkCount = 0;
3 void setup() {
4     Serial.begin(9600); // 初始化串口0
5     pinMode(5, OUTPUT); // 设置5号引脚为输出模式
6     digitalWrite(5, HIGH); // 关闭LED
7 }
8
9 void loop() {
10     if (Serial.available() > 0) {
11         receivedChar = Serial.read();
12         // 检查是否为有效数字字符 (1-9)
13         if (receivedChar >= '1' && receivedChar <= '9') {
14             blinkCount = receivedChar - '0'; // 将字符转换为数字
15
16             // 根据输入的数字控制LED闪烁次数
17             for (int i = 0; i < blinkCount; i++) {
18                 digitalWrite(5, LOW);
19                 delay(500);
20                 digitalWrite(5, HIGH);
21                 delay(500);
22             }
23         }
24         else {
25             // 输入无效时提示错误信息
26             Serial.println("Error: Please enter a number between 1 and 9.");
27         }
28     }
29 }
30
```

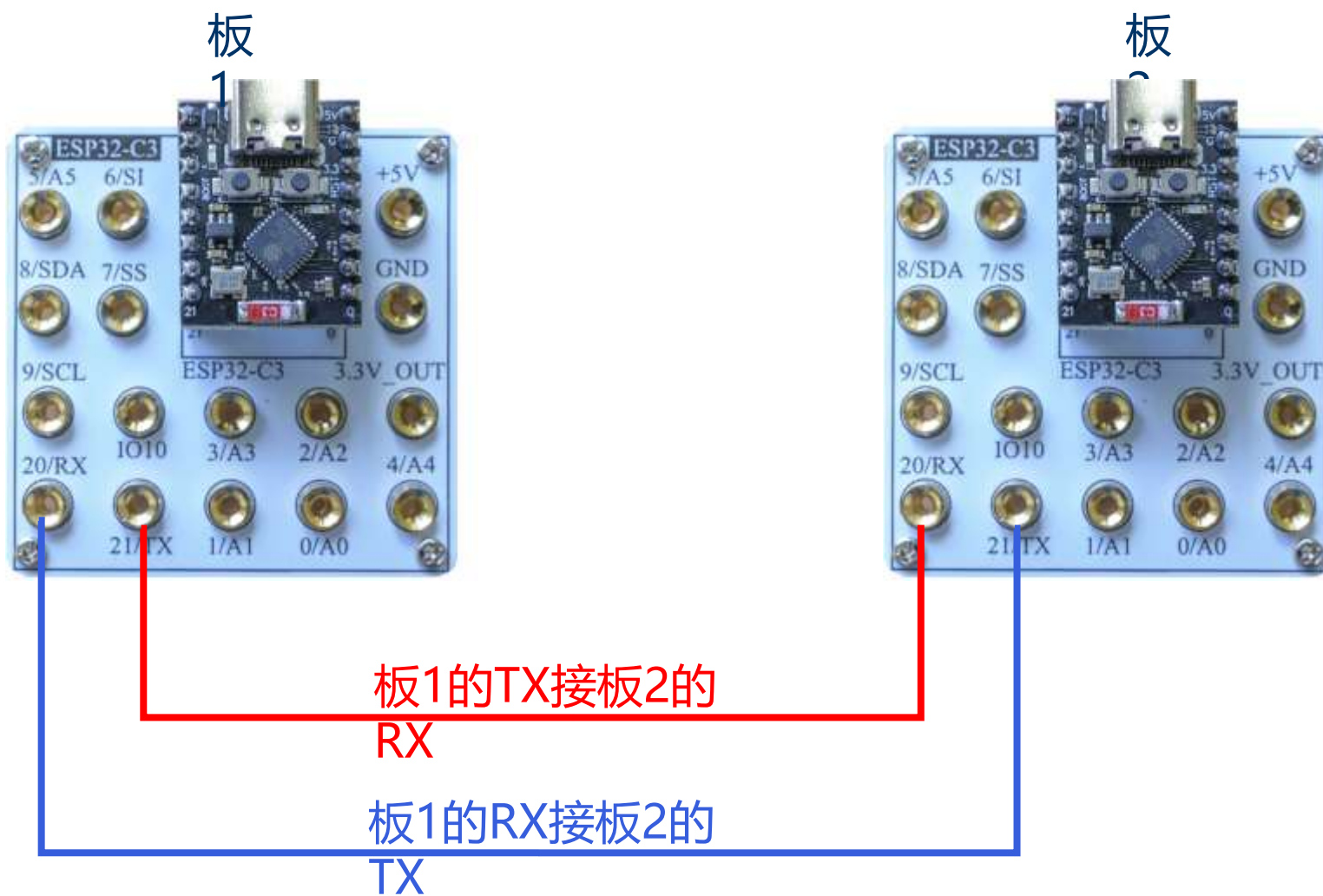


实验5：开发板与开发板通信

■ 实验内容

- 将两块开发板的串口连接在一起
 - 开发板1的TX--开发板2的RX
 - 开发板1的RX--开发板2的TX
- 开发板1作为主机从电脑端接收数据并将数据转发到开发板2，开发板1再将从开发板2接收到的数据转发回电脑端
- 开发板2作为从机通过串口接收开发板1发送的数据并且原封不动的转发回开发板1







实验5：开发板与开发板通信

■ 开发板1代码

```
1  #define UART_RX_Pin 20
2  #define UART_TX_Pin 21
3  String comdata = "";
4
5  void setup()
6  {
7      Serial.begin(9600); //初始化串口0
8      Serial1.begin(9600, SERIAL_8N1, UART_RX_Pin, UART_TX_Pin); //初始化串口1
9  }
10
11 void loop()
12 {
13     while(Serial.available() > 0) //开发板1接收电脑发送的字符串
14     {
15         comdata = comdata + char(Serial.read());
16     }
17     if(comdata.length() > 0)
18     {
19         Serial.print("Data from compute is:");
20         Serial.println(comdata);
21         Serial.println("The data is sent to R2.");
22         Serial1.print(comdata);
23         comdata.remove(0);
24     }
25     while(Serial1.available() > 0) //开发板1接收开发板2发送的字符串并转发给电脑
26     {
27         comdata = comdata + char(Serial1.read());
28     }
29     if(comdata.length() > 0)
30     {
31         Serial.print("I have received message from R2:");
32         Serial.println(comdata);
33         comdata.remove(0);
34     }
35     delay(200);
36 }
```



实验5：开发板与开发板通信

■ 开发板2代码

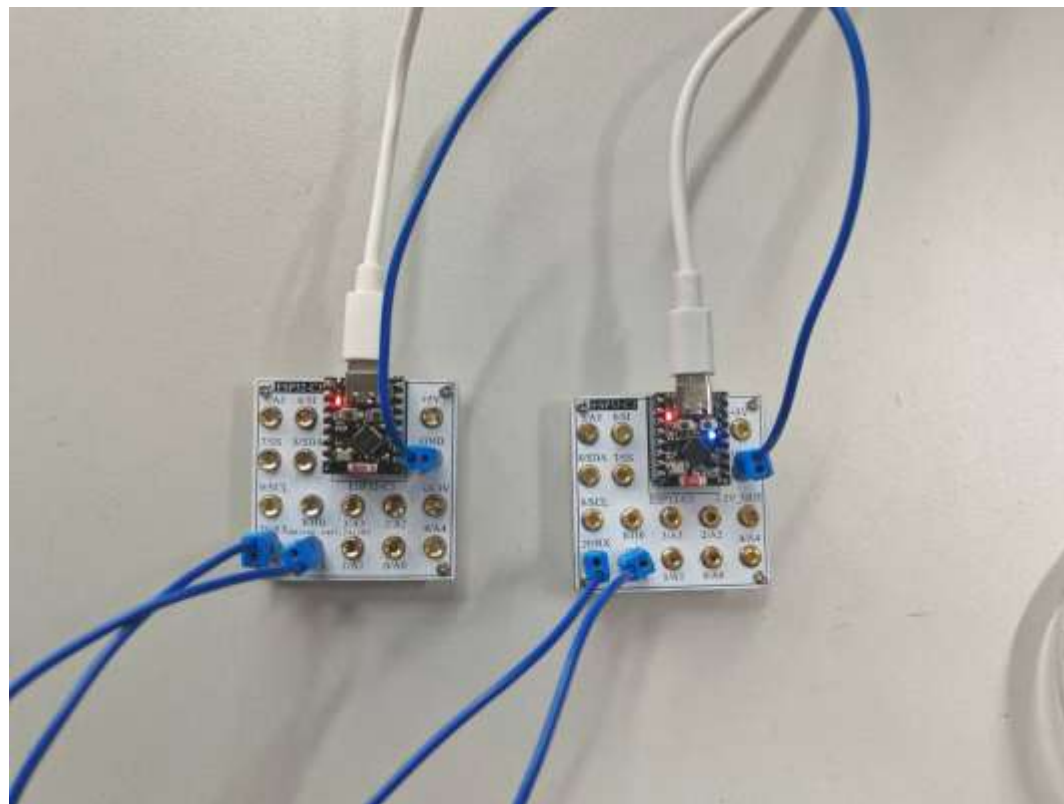
```
1  #define UART_RX_Pin 20
2  #define UART_TX_Pin 21
3  String comdata = "";
4
5  void setup()
6  {
7      Serial1.begin(9600,SERIAL_8N1,UART_RX_Pin,UART_TX_Pin);//初始化串口1
8  }
9
10 void loop()
11 {
12     //串口1接收开发板发送的字符串并原封不动转发回去
13     while(Serial1.available()>0)
14     {
15         comdata = comdata+char(Serial1.read());
16     }
17     if(comdata.length()>0)
18     {
19         Serial1.print(comdata);
20         comdata.remove(0);
21     }
22     delay(200);
23 }
```



实验6（选做）：串口对话聊天机

■ 实验内容

- 将两块开发板的串口连接在一起
 - 开发板1的TX--开发板2的RX
 - 开发板1的RX--开发板2的TX
- 两个开发板均可从电脑端接收数据并将数据转发到对端开发板，也可将对端开发板发送的数据转发给电脑
- 使得两台电脑可以开发板串口互相通信
- 请同学们自行编写相关代码





谢谢！